CARCI

Cost Anomaly Root Cause Intelligence

Complete Code Documentation

A teaching guide to every file and every line

AWS Cost Anomaly Detection using Isolation Forest

Python · scikit-learn · SHAP · Streamlit · Google Gemini

Dataset: 10,000 rows · F1 score 0.751

Contents

1. What this system does
    - 1.1 The architecture
    - 1.2 Every file at a glance
2. The concepts you need first
    - 2.1 Supervised vs unsupervised
    - 2.2 Isolation Forest explained
    - 2.3 Feature engineering
    - 2.4 The four metrics
3. app.py — The user interface
    - 3.1 Purpose and concepts
    - 3.2 Complete source code
4. carci/schema.py — Finding the right columns
    - 4.1 Purpose and concepts
    - 4.2 Complete source code
5. carci/preprocessing.py — Cleaning the data
    - 5.1 Purpose and concepts
    - 5.2 Complete source code
6. carci/shared.py — Feature engineering and data splitting
    - 6.1 Purpose and concepts
    - 6.2 Complete source code
7. carci/isolation_forest_model.py — The machine learning model
    - 7.1 Purpose and concepts
    - 7.2 Complete source code
8. carci/analysis.py — From scores to explanations
    - 8.1 Purpose and concepts
    - 8.2 Complete source code
9. carci/llm_reasoning.py — Writing the root cause
    - 9.1 Purpose and concepts
    - 9.2 Complete source code
10. carci/reporting.py — Building the exports
    - 10.1 Purpose and concepts
    - 10.2 Complete source code
11. How the evaluation works
    - 11.1 The results
    - 11.2 Label leakage: why 1.000 is a bug
    - 11.3 Testing your own design decisions

12. Known limitations

13. Glossary

14. Questions to test understanding

1 What this system does

CARCI — Cost Anomaly Root Cause Intelligence — reads an AWS billing file and answers three questions in order:

1. **What is unusual?** An Isolation Forest scores every line item.
2. **Why is it unusual?** SHAP attribution identifies which features drove the score.
3. **What should I do?** A language model turns those figures into a written explanation and a recommended action naming the service, day, region and resource.

Most anomaly detectors stop after step 1. Steps 2 and 3 are what make this a root cause system.

1.1 The architecture

There are two separate entry points that share the same model. Students confuse these constantly, so be explicit about the difference:

```
THE WEB APP (detection)                THE COMMAND LINE (evaluation)
app.py                                 python -m carci.isolation_forest_model
     |                                      |
     |  upload a billing file               |  needs a file WITH labels
     v                                      v
schema.py        find the columns       shared.py    split into train / test
     |                                      |
     v                                      v
preprocessing.py clean the data         isolation_forest_model.py
     |                                      |   fit on normal rows only
     v                                      |   predict on the test set
shared.py        build 7 features           v
     |                                     print Accuracy / Precision / Recall / F1
     v
isolation_forest_model.py
     |           score + SHAP
     v
analysis.py      service-day rules
     |           every figure computed here
     v
llm_reasoning.py Gemini writes the explanation
     |           template fallback if offline
     v
reporting.py     text + CSV exports
     |
     v
app.py           charts, tables, downloads
```

	The web app	The CLI
Purpose	Find anomalies in a new file	Measure how good the model is
Needs labels?	No	Yes — needs `is_anomaly`
Produces	Charts, cards, explanations	Accuracy, Precision, Recall, F1
Runs SHAP?	Yes (slow)	No (fast)
Calls the LLM?	Yes, once per analysis	No
Trains on	The uploaded file itself	80% of the normal rows

> **The most common misunderstanding**
>
> The web app produces **no accuracy score**, and it cannot. Accuracy requires knowing the right answer, and a real billing file has no `is_anomaly` column — that is the entire reason the system exists. Evaluation happens only on the labelled `dataset.csv`, from the command line.

1.2 Every file at a glance

File	Lines	Responsibility
`app.py`	794	Streamlit interface, three-page wizard. No ML at all
`carci/schema.py`	188	Work out which column means what
`carci/preprocessing.py`	66	Clean and standardise the data
`carci/shared.py`	212	Build the 7 features; split train/test
`carci/isolation_forest_model.py`	302	The model, scoring, SHAP, evaluation
`carci/analysis.py`	507	Business rules, anomaly facts, orchestration
`carci/llm_reasoning.py`	439	The Gemini call that writes the root cause text
`carci/reporting.py`	98	Text and CSV exports
`dataset.csv`	10,000 rows	Training and testing data
`requirements.txt`	12	Pinned dependencies

How to read the rest of this document

Sections 3 to 10 cover one file each. Every section gives the purpose, the concepts a student needs, a walkthrough of the important parts, and then the complete source code with line numbers, so the walkthrough can be followed against the real thing.

Those listings are not typed into this document. They are read from the source tree each time it is built by `tools/build_code_documentation.py`, so the code printed here is the code that runs. Line references in the

prose are maintained by hand and are the one part that can fall behind — if a number looks wrong, trust the listing.

2 The concepts you need first

2.1 Supervised versus unsupervised learning

	Supervised	Unsupervised
Needs labels?	Yes — you must know the answers	No
Learns	The mapping from input to known answer	The structure of the data
Example here	Gradient Boosting (comparison only)	Isolation Forest (the real model)

CARCI must be unsupervised, because nobody hand-labels a real AWS bill. The system has to identify unusual spending without ever being told what unusual looks like.

2.2 Isolation Forest in one paragraph

Pick a random feature. Pick a random value to split on. Cut the data in two. Repeat until every point sits alone. Count the cuts each point required. A point buried inside a dense cluster needs many cuts; a point far from everything is separated almost immediately. Anomalies are the easy ones to isolate. Build 1,000 such trees, average the path lengths, and you have a robust anomaly score.

2.3 Feature engineering

The model can only split on one feature at a time, so it cannot by itself discover that "cost is high relative to usage". We compute that relationship for it as a single number, `cost_per_usage`. That is feature engineering: giving the model the relationships it cannot derive on its own.

2.4 The four metrics

Given TP (correctly flagged anomalies), FP (false alarms), FN (missed anomalies) and TN (correctly ignored normals):

Metric	Formula	The question it answers	CARCI
Accuracy	(TP+TN) / all	How often is it right overall?	0.845
Precision	TP / (TP+FP)	When it raises an alarm, is it real?	0.877
Recall	TP / (TP+FN)	What fraction of anomalies did it catch?	0.657
F1	harmonic mean of P and R	The balance between the two	0.751

Read CARCI's numbers as a sentence: "When CARCI raises an alarm it is right about 88% of the time, but it only catches about 66% of the anomalies that exist." For a cost tool that trade-off is the right way round — a tool that cries wolf gets ignored.

Why accuracy alone is a trap

If only 5% of rows are anomalies, a model that flags nothing at all scores 95% accuracy. It is also completely useless. This is why precision and recall must always be reported alongside accuracy.

3 `app.py` — The user interface

`app.py`

794 lines · The user interface

3.1 Purpose and concepts

What it is: the entire Streamlit web application — upload box, buttons, charts, tables, expanders and download buttons. It is the only file a user ever interacts with directly.

What it is not: it contains no machine learning whatsoever. Not one line of model code. Every ML operation happens behind the single call to `build_analysis_result()` on line 253.

> **Teaching point**
>
> This is the clearest example of *separation of concerns* in the project. You could delete this file and replace Streamlit with a Flask API, a desktop app, or a command-line tool, and the detection logic would not change by a single character. When you design software, ask yourself: "if I changed the interface, how much else would break?" Here, the answer is nothing.

The three-page wizard

The interface is a wizard, not a single scrolling page. `PAGES` at lines 217–221 declares the three steps, and `st.session_state.page` records which one the user is on:

Page	Function	What it shows
1 — Upload CSV	`_show_upload_page()` (line 716)	File uploader, Start Analysis, error messages
2 — Anomaly Summary	`_show_summary_page()` (line 633)	Four metric tiles, daily trend chart, impact-by-service bar chart
3 — Investigation Details	`_show_investigation_page()` (line 661)	Anomaly table, cost breakdowns, root cause cards, exports

`_goto()` at line 296 is how navigation happens: it writes the new page number into session state and calls `st.rerun()`. Streamlit has no router — the whole script re-executes and the `if` chain at lines 785–790 draws whichever page the state names.

How the page flows

`main()` at line 766 is the whole application. Read it first — everything else is a helper it calls:

1. Inject the CSS (line 767).
2. Set up four *session state* variables (769–776). Streamlit re-runs the entire script on every click, so ordinary Python variables would be wiped. Session state is how a value survives from one click to the next.

3. **Guard the wizard** (779–780). Pages 2 and 3 have nothing to show without a result, so a missing result forces the user back to page 1. Without this, a cleared analysis would leave the app rendering a page whose data no longer exists.
4. Draw the header and the three-card step indicator (782–783).
5. Route to the current page (785–790).

Inside page 1, the sequence is the familiar one: draw the uploader (724–728), detect a file change (734–737), draw the two buttons (741–747), then run the analysis and advance to page 2 (749–751).

Three details worth pointing out to students

1. The caching decorator (lines 245–253). `@st.cache_data` stores the result so that re-analysing the same file is instant. Critically, it is keyed on `file_bytes` — the actual *contents* — not the filename. If it were keyed on the name, two different files both called `data.csv` would collide and the second would silently show the first one's results. The cache also means navigating between pages 2 and 3 does not re-run the model, and does not call the LLM again.

2. Detecting a file change (lines 734–737). If the user selects a different file, any results still on screen belong to the old file and must be cleared. Note the condition `current_name is not None`: returning to page 1 from page 2 or 3 re-creates the uploader empty, and that must not be mistaken for the user removing the file.

3. The order of the except blocks (lines 701–709). This is a real bug that was fixed:

```
except pd.errors.ParserError as exc:   # MUST come first
except ValueError as exc:              # ParserError is a subclass of this
except Exception as exc:               # last resort, never silent
```

`ParserError` inherits from `ValueError`. Python checks `except` clauses top to bottom and takes the first match, so if `ValueError` came first it would swallow parser errors and the user would get the wrong message for a corrupt CSV. Order your exception handlers from most specific to most general.

Where the root cause appears

`_show_anomaly_details()` at line 549 draws the "Root Cause Analysis" section on page 3, one expander per anomaly. Two things there are worth reading closely:

- Lines 555–566 print a caption naming the source of the wording — either `LLM (gemini-3.5-flash-lite)` or the deterministic template. This is the fastest way to tell at a glance whether the language model actually ran. See section 9.
- Lines 581–582 render the recommended action as its own bold line, but only in LLM mode. The template paragraph already ends with its own "Recommended action:" sentence, so printing it twice would be duplication.

Three helpers draw the investigation itself, and each one returns immediately when its section is empty — which is what makes the same card work in both LLM and template mode:

Helper	Line	Draws
`_show_headline()`	497	The suspected cause as a banner, coloured by severity
`_show_likely_causes()`	514	Ranked hypotheses, each with a confidence chip and its evidence
`_show_investigation_steps()`	540	The numbered checks that would confirm or eliminate them

`SEVERITY_COLORS` and `CONFIDENCE_COLORS` (lines 226–239) hold the palettes. `SEVERITY_MARKS` exists because the collapsed expander title is plain text — no HTML renders there — so severity is carried into it as a short punctuation mark instead of a coloured badge.

Note that `html.escape()` is applied again at lines 507, 531 and 533. The headline, cause and evidence strings are written by a language model and rendered as raw HTML, so they are untrusted input for exactly the same reason the filename is.

> **Security note**
>
> Line 309 calls `html.escape()` on the uploaded filename before displaying it. The upload box is rendered with `unsafe_allow_html=True`, and the filename is controlled by the user. Without escaping, a file named `<script>alert(1)</script>.csv` would execute code in the browser. This is a cross-site scripting (XSS) vulnerability, and escaping is the fix.

3.2 Complete source code

```
1  from __future__ import annotations
2
3  import html
4  import io
5
6  import pandas as pd
7  import plotly.express as px
8  import streamlit as st
9
10 from carci.analysis import build_analysis_result
11 from carci.reporting import build_report_dataframe
12
13
14 st.set_page_config(
15     page_title="CARCI - Cost Anomaly Root Cause Intelligence",
16     page_icon="C",
17     layout="wide",
18 )
19
20
21 PAGE_CSS = """
22 <style>
23 body {
24     background: linear-gradient(135deg, #f8fafc 0%, #ecfeff 100%);
25 }
26 header[data-testid="top-bar"],
27 .stApp {
28     background: transparent !important;
29 }
30 .carci-header {
```

```
    border-radius: 12px;
    background: linear-gradient(90deg, #047857 0%, #0f766e 100%);
    color: white;
    padding: 1.9rem 2.1rem;
    box-shadow: 0 22px 48px rgba(15, 23, 42, 0.08);
    margin-bottom: 1.4rem;
}
.carci-header h1 {
    margin: 0;
    font-size: 2.6rem;
    font-weight: 800;
    letter-spacing: 0.01em;
}
.carci-header .carci-subtitle {
    margin: 0.9rem 0 0;
    font-size: 1.02rem;
    font-weight: 700;
    color: #ffffff;
}
.carci-header .carci-tagline {
    margin: 0.45rem 0 0;
    color: #d1fae5;
    font-size: 0.95rem;
}
/* Step indicator: one card per page, the active one gets the green rail. */
.step-card {
    border: 1px solid #e2e8f0;
    border-left: 4px solid #e2e8f0;
    border-radius: 8px;
    background: #ffffff;
    padding: 0.85rem 1.1rem;
    height: 100%;
}
.step-card.active {
    border-left-color: #0f766e;
    background: #ecfdf5;
}
.step-card .step-num {
    margin: 0;
    font-size: 0.7rem;
    font-weight: 700;
    letter-spacing: 0.12em;
    text-transform: uppercase;
    color: #64748b;
}
.step-card.active .step-num {
    color: #0f766e;
}
.step-card .step-name {
    margin: 0.35rem 0 0;
    font-size: 1rem;
    font-weight: 700;
    color: #0f172a;
}
.carci-eyebrow {
    font-size: 0.76rem;
    font-weight: 700;
    letter-spacing: 0.12em;
    text-transform: uppercase;
    color: #0f766e;
    margin: 1.4rem 0 0.2rem;
}
.carci-page-title {
    margin: 0.1rem 0 0.6rem;
    font-size: 2.1rem;
```

```
    font-weight: 700;
    color: #0f172a;
    line-height: 1.2;
}
.carci-page-note {
    margin: 0 0 1.1rem;
    color: #475569;
    font-size: 0.95rem;
    max-width: 62rem;
}
.carci-card-title {
    font-size: 1.05rem;
    font-weight: 700;
    color: #0f172a;
    margin: 0.1rem 0 0.2rem;
}
.carci-card {
    border-radius: 12px;
    background: white;
    padding: 1.25rem;
    box-shadow: 0 14px 36px rgba(15, 23, 42, 0.05);
    border: 1px solid #e2e8f0;
    margin-bottom: 1rem;
}
.carci-upload-box {
    border: 2px dashed #cbd5e1;
    border-radius: 12px;
    padding: 1.5rem;
    text-align: center;
    background: #f8fafc;
    margin-bottom: 1rem;
}
.upload-label {
    margin: 0;
    color: #334155;
    font-size: 0.95rem;
}
.upload-name {
    margin: 0.4rem 0 0;
    font-size: 1.05rem;
    font-weight: 700;
    color: #0f172a;
}
.carci-small-note {
    color: #475569;
    margin-top: 0.5rem;
}
/* Metric tiles rendered as bordered cards, matching the summary page. */
div[data-testid="stMetric"] {
    background: #ffffff;
    border: 1px solid #e2e8f0;
    border-radius: 10px;
    padding: 1rem 1.15rem;
    box-shadow: 0 8px 22px rgba(15, 23, 42, 0.04);
}
div[data-testid="stMetric"] label p {
    color: #475569;
    font-size: 0.9rem;
}
div[data-testid="stMetricValue"] {
    font-size: 2rem;
    font-weight: 700;
    color: #0f172a;
}
/* Root cause card internals: headline banner, ranked hypotheses, evidence. */
```

```
.rca-headline {
    border: 1px solid;
    border-radius: 8px;
    padding: 0.65rem 0.85rem;
    margin-bottom: 0.9rem;
}
.rca-severity {
    font-size: 0.7rem;
    font-weight: 800;
    letter-spacing: 0.11em;
    margin-right: 0.6rem;
}
.rca-headline-text {
    font-size: 1.02rem;
    font-weight: 700;
    color: #0f172a;
}
.rca-cause {
    border-left: 3px solid #e2e8f0;
    padding: 0.15rem 0 0.15rem 0.75rem;
    margin: 0 0 0.7rem;
}
.rca-cause-head {
    display: flex;
    align-items: baseline;
    gap: 0.5rem;
    flex-wrap: wrap;
}
.rca-rank {
    font-size: 0.78rem;
    font-weight: 800;
    color: #94a3b8;
}
.rca-chip {
    font-size: 0.66rem;
    font-weight: 700;
    letter-spacing: 0.05em;
    text-transform: uppercase;
    padding: 0.1rem 0.45rem;
    border-radius: 999px;
    white-space: nowrap;
}
.rca-cause-text {
    font-size: 0.95rem;
    font-weight: 600;
    color: #0f172a;
}
.rca-evidence {
    margin: 0.25rem 0 0;
    font-size: 0.86rem;
    color: #475569;
}
</style>
"""


PAGES = {
    1: ("Step 1", "Upload CSV"),
    2: ("Step 2", "Anomaly Summary"),
    3: ("Step 3", "Investigation Details"),
}

# Severity and confidence are ranked words, so they are coloured on one warm-to-cool
# scale rather than by hue alone. The mark prefixes the collapsed expander title,
# where no HTML can be rendered.
```

```
SEVERITY_COLORS = {
    "critical": {"bg": "#fef2f2", "line": "#fecaca", "text": "#b91c1c"},
    "high": {"bg": "#fff7ed", "line": "#fed7aa", "text": "#c2410c"},
    "medium": {"bg": "#fefce8", "line": "#fde68a", "text": "#a16207"},
    "low": {"bg": "#f8fafc", "line": "#e2e8f0", "text": "#475569"},
}
SEVERITY_MARKS = {"critical": "!!!", "high": "!!", "medium": "!", "low": "·"}

CONFIDENCE_COLORS = {
    "high": {"bg": "#dcfce7", "text": "#15803d"},
    "medium": {"bg": "#fef9c3", "text": "#a16207"},
    "low": {"bg": "#f1f5f9", "text": "#64748b"},
}


def load_uploaded_csv(uploaded_file) -> pd.DataFrame:
    return pd.read_csv(uploaded_file, skipinitialspace=True)


@st.cache_data(show_spinner=False, max_entries=4)
def analyze_csv_bytes(file_bytes: bytes) -> dict:
    """Run the pipeline against raw upload bytes.

    Keyed on the bytes themselves, so re-running the same file is instant and
    two different uploads never collide in the cache.
    """
    raw_df = pd.read_csv(io.BytesIO(file_bytes), skipinitialspace=True)
    return build_analysis_result(raw_df)


def _show_header() -> None:
    st.markdown(
        """
        <div class="carci-header">
            <h1>CARCI</h1>
            <p class="carci-subtitle">Cost Anomaly Root Cause Intelligence</p>
            <p class="carci-tagline">AWS billing anomaly detection, root-cause analysis, and recommended
cost actions.</p>
        </div>
        """,
        unsafe_allow_html=True,
    )


def _show_steps(current_page: int) -> None:
    """Three-card progress strip; the current page's card is highlighted."""
    columns = st.columns(len(PAGES))
    for column, (number, (step_label, step_name)) in zip(columns, PAGES.items()):
        state = "step-card active" if number == current_page else "step-card"
        column.markdown(
            f"""
            <div class="{state}">
                <p class="step-num">{step_label}</p>
                <p class="step-name">{step_name}</p>
            </div>
            """,
            unsafe_allow_html=True,
        )


def _show_section_title(eyebrow: str, title: str, note: str) -> None:
    st.markdown(
        f"""
        <p class="carci-eyebrow">{eyebrow}</p>
        <p class="carci-page-title">{title}</p>
```

```
        <p class="carci-page-note">{note}</p>
        """,
        unsafe_allow_html=True,
    )


def _goto(page: int) -> None:
    st.session_state.page = page
    st.rerun()


def _reset_analysis() -> None:
    st.session_state.analysis_result = None
    st.session_state.analysis_error = None


def _show_upload_state(uploaded_file) -> None:
    if uploaded_file:
        # Escaped: the filename is user-controlled and this block renders as raw HTML.
        safe_name = html.escape(uploaded_file.name)
        st.markdown(
            f"""
            <div class='carci-upload-box'>
                <p class='upload-label'>Selected file</p>
                <p class='upload-name'>{safe_name}</p>
                <p class='carci-small-note'>Ready to analyze. Click Start Analysis to continue.</p>
            </div>
            """,
            unsafe_allow_html=True,
        )
    else:
        st.markdown(
            """
            <div class='carci-upload-box'>
                <p class='upload-label'>No file selected yet</p>
                <p class='upload-name'>Upload an AWS CUR-style CSV file to begin.</p>
            </div>
            """,
            unsafe_allow_html=True,
        )


def _anomaly_table(result: dict) -> pd.DataFrame:
    rows = result["service_comparison"]
    if not rows:
        return pd.DataFrame()

    table = pd.DataFrame(rows)
    # Costs are now service-day totals, so the headers say so explicitly.
    table["Status"] = [
        "Reference only" if record["is_fallback"] else "Detected"
        for record in rows
    ]
    table = table.rename(
        columns={
            "service_name": "Service",
            "date": "Date",
            "anomaly_value": "Service Day Cost",
            "driver_row_cost": "Driver Row Cost",
            "normal_value": "Normal Daily Cost",
            "impact_value": "Impact",
            "percentage": "Increase %",
            "region": "Region",
            "resource": "Resource",
            "tag": "Tag",
```

```
            "usage_type": "Usage Type",
            "model_anomaly_score": "Anomaly Score",
            "model_flag": "Model Flag",
        }
    )
    return table[
        [
            "Service",
            "Date",
            "Status",
            "Service Day Cost",
            "Driver Row Cost",
            "Normal Daily Cost",
            "Impact",
            "Increase %",
            "Region",
            "Resource",
            "Tag",
            "Usage Type",
            "Anomaly Score",
            "Model Flag",
        ]
    ]


def _show_metrics(result: dict) -> None:
    metadata = result["metadata"]
    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Rows analyzed", f"{metadata['row_count']:,}")
    col2.metric("Services", f"{metadata['service_count']:,}")
    col3.metric("Detected anomalies", f"{metadata['anomaly_count']:,}")
    col4.metric("Total cost", f"${metadata['total_cost']:,.2f}")
    st.caption(f"Date range: {metadata['date_range']} | Attribution: {metadata['attribution_method']}")

    if metadata["anomaly_count"] == 0 and metadata["shown_count"] > 0:
        st.warning(
            "No service-day crossed the anomaly threshold. The single entry below is the "
            "highest-impact service-day, shown for reference only."
        )
    if metadata["truncated"]:
        st.info(
            f"{metadata['total_found']} service-days qualified as anomalies. "
            f"Showing the {metadata['shown_count']} highest-priority ones."
        )


def _show_charts(result: dict, anomaly_table: pd.DataFrame) -> None:
    daily_df = pd.DataFrame(result["daily_summary"])
    left, right = st.columns(2)

    with left:
        with st.container(border=True):
            st.markdown("<p class='carci-card-title'>Daily Cost Trend</p>", unsafe_allow_html=True)
            if daily_df.empty:
                st.info("No daily cost data available.")
            else:
                fig = px.line(daily_df, x="date", y="cost", markers=True)
                fig.update_traces(
                    line={"color": "#0f766e", "width": 2},
                    marker={"size": 6, "color": "#0f766e"},
                    showlegend=False,
                )
                anomaly_days = daily_df[daily_df["has_anomaly"]]
                if not anomaly_days.empty:
                    fig.add_scatter(
```

```
                        x=anomaly_days["date"],
                        y=anomaly_days["cost"],
                        mode="markers",
                        marker={"size": 11, "color": "#dc2626"},
                        name="Anomaly day",
                    )
                fig.update_layout(
                    height=340,
                    margin={"l": 20, "r": 20, "t": 30, "b": 20},
                    xaxis_title="Date",
                    yaxis_title="Cost",
                    yaxis_tickprefix="$",
                    plot_bgcolor="white",
                    legend={
                        "orientation": "h",
                        "yanchor": "bottom",
                        "y": 1.02,
                        "xanchor": "right",
                        "x": 1,
                    },
                )
                fig.update_xaxes(showgrid=False)
                fig.update_yaxes(gridcolor="#e2e8f0")
                st.plotly_chart(fig, use_container_width=True)

    with right:
        with st.container(border=True):
            st.markdown(
                "<p class='carci-card-title'>Top Anomaly Impact by Service</p>",
                unsafe_allow_html=True,
            )
            if anomaly_table.empty:
                st.info("No anomaly rows were produced for this upload.")
            else:
                service_fig = px.bar(
                    anomaly_table.head(10),
                    x="Impact",
                    y="Service",
                    orientation="h",
                )
                service_fig.update_traces(
                    marker_color="#2f54eb",
                    texttemplate="$%{x:,.0f}",
                    textposition="outside",
                    cliponaxis=False,
                )
                service_fig.update_layout(
                    height=340,
                    margin={"l": 20, "r": 40, "t": 30, "b": 20},
                    yaxis={"categoryorder": "total ascending"},
                    xaxis_tickprefix="$",
                    plot_bgcolor="white",
                )
                service_fig.update_xaxes(showgrid=False)
                service_fig.update_yaxes(showgrid=False)
                st.plotly_chart(service_fig, use_container_width=True)


def _show_breakdowns(result: dict) -> None:
    st.subheader("Overall Cost Breakdown")
    labels = {
        "service": "Service",
        "region": "Region",
        "resource": "Resource",
        "tag": "Tag",
```

```
        "usage_type": "Usage Type",
    }
    # Stacked rather than tabbed, so every dimension is visible on one scroll.
    for key, label in labels.items():
        st.markdown(f"**{label}**")
        data = pd.DataFrame(result["overall_breakdowns"].get(key, []))
        if data.empty:
            st.info("No data available for this breakdown.")
        else:
            st.dataframe(data, use_container_width=True)


def _show_headline(record: dict, severity: str) -> None:
    """The suspected cause, as a coloured banner above the narrative."""
    headline = record.get("headline")
    if not headline:
        return
    palette = SEVERITY_COLORS.get(severity, SEVERITY_COLORS["low"])
    st.markdown(
        f"""
        <div class="rca-headline" style="background:{palette['bg']};border-color:{palette['line']}">
            <span class="rca-severity" style="color:{palette['text']}">{severity.upper()}</span>
            <span class="rca-headline-text">{html.escape(headline)}</span>
        </div>
        """,
        unsafe_allow_html=True,
    )


def _show_likely_causes(record: dict) -> None:
    """Ranked hypotheses. Empty in template mode, which is why this can be skipped."""
    causes = record.get("likely_causes") or []
    if not causes:
        return
    st.markdown("**Likely causes**")
    for rank, cause in enumerate(causes, start=1):
        confidence = cause.get("confidence", "low")
        palette = CONFIDENCE_COLORS.get(confidence, CONFIDENCE_COLORS["low"])
        st.markdown(
            f"""
            <div class="rca-cause">
                <div class="rca-cause-head">
                    <span class="rca-rank">{rank}</span>
                    <span class="rca-chip" style="background:{palette['bg']};color:{palette['text']}">
                        {confidence} confidence
                    </span>
                    <span class="rca-cause-text">{html.escape(cause.get('cause', ''))}</span>
                </div>
                <p class="rca-evidence">Evidence: {html.escape(cause.get('evidence', ''))}</p>
            </div>
            """,
            unsafe_allow_html=True,
        )


def _show_investigation_steps(record: dict) -> None:
    steps = record.get("investigation_steps") or []
    if not steps:
        return
    st.markdown("**How to confirm it**")
    for number, step in enumerate(steps, start=1):
        st.markdown(f"{number}. {step}")


def _show_anomaly_details(result: dict) -> None:
```

```
    st.subheader("Root Cause Analysis")
    if not result["service_comparison"]:
        st.info("No strong anomalies were detected in this file.")
        return

    method = result["metadata"].get("explanation_method", "deterministic template")
    if method.startswith("LLM"):
        st.caption(
            f"Explanations written by {method}. Every figure quoted is computed by the "
            "pipeline and passed to the model as JSON; the model words them and recommends "
            "the action."
        )
    else:
        st.caption(
            f"Explanations from the {method}. Set GEMINI_API_KEY in .env to have the LLM "
            "write them instead."
        )

    for record in result["service_comparison"]:
        severity = record.get("severity", "low")
        title = (
            f"{SEVERITY_MARKS.get(severity, '')} {record['service_name']} | {record['date']} "
            f"| impact ${record['impact_value']:,.2f}"
        )
        with st.expander(title, expanded=False):
            _show_headline(record, severity)
            st.write(record["root_cause_analysis"])
            _show_likely_causes(record)
            _show_investigation_steps(record)
            # The template paragraph ends with its own action line; the LLM returns
            # the action as a separate field, so it is rendered here instead.
            if record.get("explanation_method", "").startswith("LLM") and record.get("recommendation"):
                st.markdown(f"**Recommended action:** {record['recommendation']}")

            breakdown_tab, feature_tab = st.tabs(["Factor Contributions", "ML Feature Attribution"])
            with breakdown_tab:
                for label, key in [
                    ("Service", "service"),
                    ("Region", "region"),
                    ("Resource", "resource"),
                    ("Tag", "tag"),
                    ("Usage Type", "usage_type"),
                ]:
                    rows = pd.DataFrame(record["breakdowns"].get(key, []))
                    if not rows.empty:
                        st.markdown(f"**{label}**")
                        st.dataframe(rows, use_container_width=True)

            with feature_tab:
                feature_rows = pd.DataFrame(record["feature_contributions"])
                st.dataframe(feature_rows, use_container_width=True)


def _show_downloads(result: dict, anomaly_table: pd.DataFrame) -> None:
    st.subheader("Export")
    col1, col2, col3 = st.columns(3)
    col1.download_button(
        "Download Text Report",
        data=result["report_text"],
        file_name="carci_anomaly_report.txt",
        mime="text/plain",
        use_container_width=True,
    )
    col2.download_button(
        "Download Anomaly CSV",
```

```
        data=anomaly_table.to_csv(index=False).encode("utf-8"),
        file_name="carci_anomaly_table.csv",
        mime="text/csv",
        use_container_width=True,
        disabled=anomaly_table.empty,
    )
    detailed = build_report_dataframe(result)
    col3.download_button(
        "Download Detailed CSV",
        data=detailed.to_csv(index=False).encode("utf-8"),
        file_name="carci_detailed_report.csv",
        mime="text/csv",
        use_container_width=True,
        disabled=detailed.empty,
        help="Long-format export including metadata and per-feature SHAP contributions.",
    )


def _show_summary_page(result: dict, file_name: str | None) -> None:
    label = file_name or "the uploaded file"
    st.success(f"Analysis complete for **{label}**. Review the summary below.")
    _show_section_title(
        "Summary",
        "Rows analyzed, daily trend, and anomaly impact",
        "This page highlights the key signals: analyzed rows, total cost, anomaly count, "
        "daily cost movement, and the services driving the largest impact.",
    )
    _show_metrics(result)
    _show_charts(result, _anomaly_table(result))

    st.markdown("---")
    col1, col2, col3 = st.columns(3)
    if col1.button("Back to Upload", use_container_width=True, key="summary_back"):
        _goto(1)
    if col2.button(
        "Continue to Page 3: Investigation Details",
        type="primary",
        use_container_width=True,
        key="summary_next",
    ):
        _goto(3)
    if col3.button("New Analysis", use_container_width=True, key="summary_new"):
        _reset_analysis()
        _goto(1)


def _show_investigation_page(result: dict) -> None:
    _show_section_title(
        "Investigation",
        "Anomaly table, cost breakdown, and root cause",
        "Every flagged service-day with its driver row, the overall cost breakdown by "
        "dimension, and a per-anomaly explanation with ML feature attribution.",
    )

    anomaly_table = _anomaly_table(result)
    st.subheader("Anomaly Table")
    if anomaly_table.empty:
        st.info("No anomaly rows were produced for this upload.")
    else:
        st.dataframe(anomaly_table, use_container_width=True)

    _show_breakdowns(result)
    _show_anomaly_details(result)
    _show_downloads(result, anomaly_table)

```

```
    st.markdown("---")
    col1, col2, col3 = st.columns(3)
    if col1.button("Back to Summary", use_container_width=True, key="details_back"):
        _goto(2)
    if col2.button("Back to Upload", use_container_width=True, key="details_upload"):
        _goto(1)
    if col3.button("New Analysis", use_container_width=True, key="details_new"):
        _reset_analysis()
        _goto(1)


def _run_analysis(uploaded_file) -> bool:
    """Run the pipeline and record either the result or a readable error.

    Returns True when analysis succeeded, so the caller can advance the wizard.
    """
    st.session_state.analysis_error = None
    with st.spinner("Analyzing AWS CUR data and generating explanations..."):
        try:
            st.session_state.analysis_result = analyze_csv_bytes(uploaded_file.getvalue())
            return True
        # ParserError subclasses ValueError, so it must be caught first.
        except pd.errors.ParserError as exc:
            message = f"The file could not be parsed as CSV. {exc}"
        except ValueError as exc:
            # Raised by schema detection and preprocessing for unusable files.
            message = str(exc)
        # Last resort: the message is surfaced to the user, never swallowed.
        except Exception as exc:
            message = f"Unexpected {type(exc).__name__}: {exc}"

    st.session_state.analysis_error = message
    st.session_state.analysis_result = None
    return False


def _show_upload_page() -> None:
    _show_section_title(
        "Start Analysis",
        "Upload your AWS billing CSV",
        "Select the CUR-style CSV file. When analysis completes, CARCI opens the anomaly "
        "summary page automatically.",
    )

    uploaded_file = st.file_uploader(
        "Upload your AWS CUR CSV",
        type=["csv"],
        help="Upload a CUR-style CSV file exported from AWS billing or a synthetic dataset with matching
columns.",
    )
    _show_upload_state(uploaded_file)

    # Selecting a different file invalidates results still on screen from the last one.
    # Only a real selection counts: returning here from page 2 or 3 re-creates the
    # uploader empty, and that must not discard the analysis being navigated around.
    current_name = uploaded_file.name if uploaded_file else None
    if current_name is not None and st.session_state.last_file_name != current_name:
        st.session_state.last_file_name = current_name
        _reset_analysis()

    col1, col2 = st.columns([1, 1])
    # Both buttons render unconditionally so the row keeps its layout before upload.
    start_clicked = col1.button(
        "Start Analysis",
        type="primary",
```

```
        use_container_width=True,
        disabled=uploaded_file is None,
    )
    clear_clicked = col2.button("Clear Results", use_container_width=True)

    if start_clicked and uploaded_file is not None:
        if _run_analysis(uploaded_file):
            _goto(2)

    if clear_clicked:
        _reset_analysis()
        st.rerun()

    if st.session_state.analysis_error:
        st.error(f"Unable to analyze the uploaded file: {st.session_state.analysis_error}")

    if st.session_state.analysis_result:
        st.info("Analysis is ready. Continue to the anomaly summary.")
        if st.button("Continue to Page 2: Anomaly Summary", type="primary", key="upload_next"):
            _goto(2)


def main() -> None:
    st.markdown(PAGE_CSS, unsafe_allow_html=True)

    if "analysis_result" not in st.session_state:
        st.session_state.analysis_result = None
    if "analysis_error" not in st.session_state:
        st.session_state.analysis_error = None
    if "last_file_name" not in st.session_state:
        st.session_state.last_file_name = None
    if "page" not in st.session_state:
        st.session_state.page = 1

    # Pages 2 and 3 have nothing to show without a result, so fall back to upload.
    if st.session_state.analysis_result is None:
        st.session_state.page = 1

    _show_header()
    _show_steps(st.session_state.page)

    if st.session_state.page == 2:
        _show_summary_page(st.session_state.analysis_result, st.session_state.last_file_name)
    elif st.session_state.page == 3:
        _show_investigation_page(st.session_state.analysis_result)
    else:
        _show_upload_page()


if __name__ == "__main__":
    main()
```

4 `carci/schema.py` — Finding the right columns

`carci/schema.py`

188 lines · Finding the right columns

4.1 Purpose and concepts

The problem it solves: a real AWS Cost and Usage Report does not have a column called `cost`. It has `line_item_unblended_cost`. Or `lineitemunblendedcost`. Or `net_unblended_cost`. A student dataset might just say `amount`. If the rest of the program had to handle all of these, every single file would be full of if-statements.

This file solves it once. It maps roughly ninety possible real-world column names onto eleven logical fields.

The three-stage matching strategy

`detect_columns()` (line 155) tries progressively looser matching:

1. **Normalise (line 127).** `_normalize_name()` lowercases and strips every character that is not alphanumeric or underscore. So "Line Item Unblended Cost", "line-item-unblended-cost" and "LineItemUnblendedCost" all collapse to the same string.
2. **Exact alias lookup (lines 159–162).** Check the normalised name against the known alias list.
3. **Token fallback (line 131).** If nothing matched, look for a substring. Any column whose name contains "cost", "unblended", "blended", "amount" or "ondemand" is accepted as the cost column. This is what lets the system handle a CSV it has never seen before.

Required versus optional

Only three fields are required: `date`, `service` and `cost`. Without those, cost anomaly detection is meaningless, so `find_column()` raises a `ValueError` listing the columns it did find — a helpful error, not a crash. Everything else passes `required=False` and returns `None` when absent.

> **Teaching point**
>
> `NormalizedColumns` (line 9) is a `@dataclass(frozen=True)`. "Frozen" means immutable — once created it cannot be modified. This is deliberate: the detected column mapping is a fact about the uploaded file, and nothing downstream should ever be able to change it by accident.

4.2 Complete source code

```
1  from __future__ import annotations
2
3  from dataclasses import dataclass
4  from typing import Dict, Iterable, Optional
5
6  import pandas as pd
7
```

```
8
9  @dataclass(frozen=True)
10 class NormalizedColumns:
11     date: str
12     service: str
13     region: Optional[str]
14     resource: Optional[str]
15     usage_type: Optional[str]
16     tag: Optional[str]
17     cost: str
18     usage_quantity: Optional[str]
19     autoscaling_count: Optional[str]
20     data_transfer_gb: Optional[str]
21     is_anomaly: Optional[str]
22
23
24 COLUMN_ALIASES: Dict[str, Iterable[str]] = {
25     "date": [
26         "date",
27         "usage_date",
28         "usage start date",
29         "billing period start date",
30         "bill_billing_period_start_date",
31         "bill_billingperiodstartdate",
32         "identity_time_interval",
33         "identity_timeinterval",
34         "line_item_usage_start_date",
35         "line_item_usage_end_date",
36         "lineitemusagestartdate",
37         "lineitemusageenddate",
38         "usage_start_date",
39         "time_interval",
40     ],
41     "service": [
42         "service",
43         "servicecode",
44         "product_service_code",
45         "product_servicecode",
46         "product_product_name",
47         "product_productname",
48         "line_item_product_code",
49         "lineitemproductcode",
50     ],
51     "region": [
52         "region",
53         "product_region_code",
54         "product_region",
55         "product_location",
56         "availability_zone",
57         "line_item_availability_zone",
58     ],
59     "resource": [
60         "resource",
61         "resource_id",
62         "line_item_resource_id",
63         "lineitemresourceid",
64     ],
65     "usage_type": [
66         "usage_type",
67         "line_item_usage_type",
68         "lineitemusagetype",
69         "usagetype",
70     ],
71     "tag": [
72         "tag",
```

```
        "tags",
        "resource_tags",
        "resource_tags_user_name",
        "resource_tags_user_environment",
        "resource_tags_user_project",
        "tag_name",
        "tag_project",
        "tag_environment",
        "tag-project / tag-name",
        "project_name",
        "environment",
    ],
    "cost": [
        "cost",
        "amount",
        "net_unblended_cost",
        "pricing_public_on_demand_cost",
        "line_item_unblended_cost",
        "line_item_blended_cost",
        "line_item_net_unblended_cost",
        "lineitemunblendedcost",
        "lineitemblendedcost",
        "lineitemnetunblendedcost",
        "unblended_cost",
    ],
    "usage_quantity": [
        "usage_quantity",
        "usage amount",
        "line_item_usage_amount",
        "lineitemusageamount",
        "usageamount",
    ],
    "autoscaling_count": [
        "autoscaling_count",
        "autoscaling-count",
        "autoscalingcount",
        "auto_scaling_count",
    ],
    "data_transfer_gb": [
        "data_transfer_gb",
        "data-transfer-gb",
        "datatransfergb",
        "transfer_gb",
        "network_gb",
    ],
    "is_anomaly": [
        "is_anomaly",
        "label",
        "anomaly",
        "is_anomalous",
    ],
}


def _normalize_name(name: str) -> str:
    return "".join(ch.lower() for ch in str(name).strip() if ch.isalnum() or ch == "_")


def _fallback_match(normalized_map: Dict[str, str], logical_name: str) -> Optional[str]:
    candidates = list(normalized_map.items())

    token_map = {
        "date": ("date", "timeinterval", "time_interval", "usagestart", "billingperiodstart"),
        "cost": ("cost", "unblended", "blended", "amount", "ondemand"),
        "service": ("service", "productcode", "productname"),
```

```
        "region": ("region", "location", "availabilityzone"),
        "resource": ("resource", "instance", "arn"),
        "usage_type": ("usagetype", "operation", "lineitemtype"),
        "tag": ("tag", "project", "environment"),
        "usage_quantity": ("usagequantity", "usageamount"),
        "autoscaling_count": ("autoscalingcount", "autoscaling"),
        "data_transfer_gb": ("datatransfergb", "transfergb", "networkgb"),
        "is_anomaly": ("isanomaly", "label", "anomaly"),
    }

    for normalized, original in candidates:
        if any(token in normalized for token in token_map[logical_name]):
            return original

    return None


def detect_columns(df: pd.DataFrame) -> NormalizedColumns:
    normalized_map = {_normalize_name(col): col for col in df.columns}

    def find_column(logical_name: str, required: bool = True) -> Optional[str]:
        for alias in COLUMN_ALIASES[logical_name]:
            matched = normalized_map.get(_normalize_name(alias))
            if matched:
                return matched

        fallback = _fallback_match(normalized_map, logical_name)
        if fallback:
            return fallback

        if required:
            available_columns = ", ".join(str(col) for col in df.columns)
            raise ValueError(
                f"Required column for '{logical_name}' was not found in the uploaded CSV. "
                f"Available columns: {available_columns}"
            )
        return None

    return NormalizedColumns(
        date=find_column("date"),
        service=find_column("service"),
        region=find_column("region", required=False),
        resource=find_column("resource", required=False),
        usage_type=find_column("usage_type", required=False),
        tag=find_column("tag", required=False),
        cost=find_column("cost"),
        usage_quantity=find_column("usage_quantity", required=False),
        autoscaling_count=find_column("autoscaling_count", required=False),
        data_transfer_gb=find_column("data_transfer_gb", required=False),
        is_anomaly=find_column("is_anomaly", required=False),
    )
```

5 `carci/preprocessing.py` — Cleaning the data

carci/preprocessing.py

66 lines · Cleaning the data

5.1 Purpose and concepts

Its one job: take the messy uploaded file plus the column mapping from `schema.py`, and return one predictable DataFrame with fixed column names and clean data types.

Look at the shape of `preprocess_cur_data()`. It builds a brand-new DataFrame rather than modifying the old one, and every single field follows the same pattern:

```
"region": (
    df[cols.region].fillna("Unknown Region").astype(str)
    if cols.region        # did schema.py find this column?
    else "Unknown Region" # no - use a safe default for every row
),
```

This is the mechanism behind "missing optional columns produce a partial explanation rather than an error". A file with no region column still analyses perfectly; the explanations simply say "Unknown Region".

The three cleaning rules

Line	Rule	Why
13	`errors="coerce"` on dates	Unparseable dates become `NaT` instead of throwing an exception
59	`dropna(subset=["date"])`	A row with no valid date cannot be placed on a timeline, so it is removed
61	`cost >= 0`	Negative costs are AWS credits and refunds, not spending anomalies

Line 63 raises a clear error if nothing survives, rather than letting an empty DataFrame cause a confusing crash five files later.

> **Teaching point**
>
> `errors="coerce"` appears on every numeric and date conversion. It means "if this value cannot be converted, produce a null instead of raising". For user-uploaded data this is almost always what you want: one malformed cell in row 4,000 should not destroy the entire analysis.

5.2 Complete source code

```
from __future__ import annotations

import pandas as pd

from carci.schema import detect_columns
```

```python


def preprocess_cur_data(df: pd.DataFrame) -> pd.DataFrame:
    cols = detect_columns(df)

    prepared = pd.DataFrame(
        {
            "date": pd.to_datetime(df[cols.date], errors="coerce").dt.date,
            "service": df[cols.service].fillna("Unknown Service").astype(str),
            "region": (
                df[cols.region].fillna("Unknown Region").astype(str)
                if cols.region
                else "Unknown Region"
            ),
            "resource": (
                df[cols.resource].fillna("Unknown Resource").astype(str)
                if cols.resource
                else "Unknown Resource"
            ),
            "usage_type": (
                df[cols.usage_type].fillna("Unknown Usage Type").astype(str)
                if cols.usage_type
                else "Unknown Usage Type"
            ),
            "tag": (
                df[cols.tag].fillna("Untagged").astype(str)
                if cols.tag
                else "Untagged"
            ),
            "cost": pd.to_numeric(df[cols.cost], errors="coerce").fillna(0.0),
            "usage_quantity": (
                pd.to_numeric(df[cols.usage_quantity], errors="coerce").fillna(0.0)
                if cols.usage_quantity
                else 0.0
            ),
            "autoscaling_count": (
                pd.to_numeric(df[cols.autoscaling_count], errors="coerce").fillna(0.0)
                if cols.autoscaling_count
                else 0.0
            ),
            "data_transfer_gb": (
                pd.to_numeric(df[cols.data_transfer_gb], errors="coerce").fillna(0.0)
                if cols.data_transfer_gb
                else 0.0
            ),
            "is_anomaly": (
                pd.to_numeric(df[cols.is_anomaly], errors="coerce").fillna(0).astype(int)
                if cols.is_anomaly
                else 0
            ),
        }
    )

    prepared = prepared.dropna(subset=["date"]).copy()
    prepared["date"] = pd.to_datetime(prepared["date"])
    prepared = prepared[prepared["cost"] >= 0].copy()

    if prepared.empty:
        raise ValueError("The uploaded CSV does not contain valid date/cost rows after preprocessing.")

    return prepared
```

6 `carci/shared.py` — Feature engineering and data splitting

`carci/shared.py`

212 lines · Feature engineering and data splitting

6.1 Purpose and concepts

This file answers the most important question in any ML project: **what numbers does the model actually look at?** The answer is `FEATURE_COLUMNS` at line 10 — exactly seven values per row.

Feature	Where it comes from	What it tells the model
`cost`	raw	How much this line item cost
`usage_quantity`	raw	How much was used
`autoscaling_count`	raw	How much scaling activity
`data_transfer_gb`	raw	How much network transfer
`cost_per_usage`	`cost / (usage_quantity + 1)`	Unit price. Catches "paid a lot for very little"
`transfer_per_cost`	`data_transfer_gb / (cost + 1)`	Transfer efficiency. Catches unusual data movement
`daily_z`	`add_daily_context()`	Whether this row sits on a quiet day or a spike day

> **Why the + 1 in the denominators?**
>
> If `usage_quantity` is 0, then `cost / usage_quantity` is a division by zero, producing infinity, which breaks the scaler and then the model. Adding 1 is a standard trick that guarantees a finite result while barely affecting large values. This is worth pointing out to students — the maths must survive every possible input, not just typical ones.

Engineered features: the interesting part

The first four features are copied straight from the file. The last three are *engineered* — computed by combining other columns. This matters because an Isolation Forest splits on one feature at a time. It cannot discover the relationship "cost is high relative to usage" on its own. By computing `cost_per_usage` for it, we hand it that relationship as a single number it can split on directly.

`daily_z` (lines 91–118) deserves special attention. Every other feature is row-local — it describes one line item in isolation. But the project's goal is detecting unusual *daily* spending. So `add_daily_context()` totals the cost of every calendar day, converts those totals to z-scores, and gives every row on a given day the same value. Note lines 99–113: if there is no date column, or only one day, or zero variance, it degrades safely to 0.0 rather than crashing.

The three split functions — do not confuse them

Function	Used by	What it does
`prepare_train_test_features` (line 141)	CLI evaluation	Trains on 80% of normal rows only; tests on the other 20% of normals plus every anomaly
`prepare_inference_features` (line 185)	The web app	Trains and scores on the same upload. Excludes labelled anomalies from training if a label column exists
`prepare_external_test_features` (line 170)	Two-file CLI mode	Trains on file A, tests on all of file B — the cleanest setup

> **The single most important idea in this file**
>
> Look at lines 148–159. The training set contains **only normal rows**. The model never sees a single anomaly during training. It learns what normal looks like, then meets all the anomalies for the first time at test. This is what makes the reported scores honest. If anomalies were in the training set, the model could memorise them and the results would be meaningless.

Notice what is missing

`usage_type` is **not** in `FEATURE_COLUMNS`. In this dataset that column happens to give away the answer exactly. Because the Isolation Forest never sees it, the Isolation Forest results are trustworthy. See section 11 for the full story.

6.2 Complete source code

```
1  from __future__ import annotations
2
3  from typing import Iterable, List, Optional, Tuple
4
5  import pandas as pd
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8
9
10 FEATURE_COLUMNS = [
11     "cost",
12     "usage_quantity",
13     "autoscaling_count",
14     "data_transfer_gb",
15     "cost_per_usage",
16     "transfer_per_cost",
17     # Day-level spend context. The SRS asks for detection of unusual changes in
18     # *daily* spending, but every other feature is row-local, leaving the model
19     # blind to whether a row sits on a quiet day or a spike day.
20     "daily_z",
21 ]
22
23 # Column names searched when deriving daily context, in priority order.
24 DATE_COLUMN_CANDIDATES = ["date", "billing_date", "usage_date", "usage_start_date"]
25
26 COLUMN_ALIASES = {
27     "cost_usd": "cost",
28     "label": "is_anomaly",
```

```
    "autoscaling-count": "autoscaling_count",
    "data-transfer-gb": "data_transfer_gb",
}

OPTIONAL_NUMERIC_COLUMNS = [
    "autoscaling_count",
    "data_transfer_gb",
]

REQUIRED_COLUMNS = [
    "cost",
    "usage_quantity",
    "is_anomaly",
]


def normalize_dataset_columns(df: pd.DataFrame, require_labels: bool = True) -> pd.DataFrame:
    df = df.copy()

    for source_column, target_column in COLUMN_ALIASES.items():
        if target_column not in df.columns and source_column in df.columns:
            df[target_column] = df[source_column]

    if "usage_quantity" not in df.columns:
        df["usage_quantity"] = 0
    if "is_anomaly" not in df.columns:
        df["is_anomaly"] = 0

    for column in OPTIONAL_NUMERIC_COLUMNS:
        if column not in df.columns:
            df[column] = 0

    required_columns = list(REQUIRED_COLUMNS)
    if not require_labels:
        required_columns.remove("is_anomaly")

    missing_columns = [column for column in required_columns if column not in df.columns]
    if missing_columns:
        available_columns = ", ".join(df.columns)
        missing_text = ", ".join(missing_columns)
        raise ValueError(
            f"Missing required column(s): {missing_text}. "
            f"Available columns: {available_columns}"
        )

    numeric_columns = ["cost", "usage_quantity", "is_anomaly"] + OPTIONAL_NUMERIC_COLUMNS
    for column in numeric_columns:
        df[column] = pd.to_numeric(df[column], errors="coerce").fillna(0)

    return df


def _find_date_column(df: pd.DataFrame) -> Optional[str]:
    for candidate in DATE_COLUMN_CANDIDATES:
        if candidate in df.columns:
            return candidate
    for column in df.columns:
        if "date" in str(column).lower():
            return column
    return None


def add_daily_context(df: pd.DataFrame) -> pd.DataFrame:
    """Add `daily_z`: how unusual this row's calendar day is by total spend.

```

```
    Every row on a given day shares the value, so it tells the model whether a
    line item sits on a quiet day or a spike day. Degrades to 0.0 when the file
    has no usable date column or only a single day, per SRS section 4.
    """
    df = df.copy()
    date_column = _find_date_column(df)
    if date_column is None:
        df["daily_z"] = 0.0
        return df

    parsed = pd.to_datetime(df[date_column], errors="coerce")
    if parsed.notna().sum() == 0:
        df["daily_z"] = 0.0
        return df

    daily_totals = df.assign(_day=parsed).groupby("_day", dropna=True)["cost"].sum()
    spread = float(daily_totals.std(ddof=0))
    if len(daily_totals) < 2 or spread == 0:
        df["daily_z"] = 0.0
        return df

    centre = float(daily_totals.mean())
    z_by_day = (daily_totals - centre) / spread
    df["daily_z"] = parsed.map(z_by_day).astype(float).fillna(0.0).to_numpy()
    return df


def add_engineered_features(df: pd.DataFrame, require_labels: bool = True) -> pd.DataFrame:
    df = normalize_dataset_columns(df, require_labels=require_labels)

    df["cost_per_usage"] = df["cost"] / (df["usage_quantity"] + 1)
    df["transfer_per_cost"] = df["data_transfer_gb"] / (df["cost"] + 1)
    df = add_daily_context(df)

    return df


def prepare_features(df: pd.DataFrame):
    df = add_engineered_features(df)

    x = df[FEATURE_COLUMNS]
    scaler = StandardScaler()
    x_scaled = scaler.fit_transform(x)

    return x_scaled


def prepare_train_test_features(
    df: pd.DataFrame,
    test_size: float = 0.2,
    random_state: int = 42,
):
    df = add_engineered_features(df)

    normal_df = df[df["is_anomaly"] == 0]
    anomaly_df = df[df["is_anomaly"] == 1]

    normal_train_df, normal_test_df = train_test_split(
        normal_df,
        test_size=test_size,
        random_state=random_state,
        shuffle=True,
    )

    train_df = normal_train_df.reset_index(drop=True)
```

```
    test_df = pd.concat([normal_test_df, anomaly_df], ignore_index=True)
    test_df = test_df.sample(frac=1, random_state=random_state).reset_index(drop=True)

    scaler = StandardScaler()
    x_train = scaler.fit_transform(train_df[FEATURE_COLUMNS])
    x_test = scaler.transform(test_df[FEATURE_COLUMNS])
    y_test = test_df["is_anomaly"]

    return x_train, x_test, y_test, train_df, test_df


def prepare_external_test_features(train_source_df: pd.DataFrame, test_source_df: pd.DataFrame):
    train_source_df = add_engineered_features(train_source_df)
    test_df = add_engineered_features(test_source_df)

    train_df = train_source_df[train_source_df["is_anomaly"] == 0].reset_index(drop=True)
    test_df = test_df.reset_index(drop=True)

    scaler = StandardScaler()
    x_train = scaler.fit_transform(train_df[FEATURE_COLUMNS])
    x_test = scaler.transform(test_df[FEATURE_COLUMNS])
    y_test = test_df["is_anomaly"]

    return x_train, x_test, y_test, train_df, test_df


def prepare_inference_features(
    train_df: pd.DataFrame,
    score_df: Optional[pd.DataFrame] = None,
    normal_labels: Iterable[int] = (0,),
) -> Tuple[object, object, pd.DataFrame, pd.DataFrame]:
    train_features = add_engineered_features(train_df, require_labels=False)
    score_features = add_engineered_features(score_df if score_df is not None else train_df,
require_labels=False)

    if "is_anomaly" in train_features.columns:
        normal_train_df = train_features[train_features["is_anomaly"].isin(list(normal_labels))]
        if len(normal_train_df) >= 2:
            train_features = normal_train_df

    scaler = StandardScaler()
    x_train = scaler.fit_transform(train_features[FEATURE_COLUMNS])
    x_score = scaler.transform(score_features[FEATURE_COLUMNS])

    return x_train, x_score, train_features, score_features


def print_split_summary(train_df: pd.DataFrame, test_df: pd.DataFrame) -> None:
    test_normal_count = int((test_df["is_anomaly"] == 0).sum())
    test_anomaly_count = int((test_df["is_anomaly"] == 1).sum())

    print("Training:", len(train_df), "normal records only")
    print("Testing: ", len(test_df), "records total")
    print("         ", test_normal_count, "held-out normal records")
    print("         ", test_anomaly_count, "anomaly records")
```

7 `carci/isolation_forest_model.py` — The machine learning model

`carci/isolation_forest_model.py`

302 lines · The machine learning model

7.1 Purpose and concepts

This is the heart of the system. It contains the model, the scoring function used by the app, and the evaluation routine used from the command line.

How Isolation Forest works — explain it this way

Most anomaly detection algorithms build a picture of "normal" and measure distance from it. Isolation Forest does the opposite, and the intuition is beautifully simple:

Pick a random feature. Pick a random split point. Cut the data in two. Repeat until every point is alone in its own box. Now count how many cuts each point needed.

A point in the middle of a dense cluster is surrounded by neighbours, so it takes many cuts to isolate. A point far away from everything gets separated almost immediately. Anomalies are the points that are *easy to isolate*. Fewer cuts means more anomalous.

The forest builds 1,000 such random trees and averages the path length over all of them. `decision_function()` returns that averaged score: low means anomalous, high means normal.

The model configuration (lines 52–59)

Parameter	Value	Meaning
`n_estimators`	1000	Number of random trees. More trees means a more stable score but slower training
`contamination`	0.05	The assumed proportion of anomalies. This sets where the cut-off lands
`max_samples`	1.0	Each tree sees 100% of the training rows
`random_state`	123	Fixes the randomness so results reproduce exactly every run

> **contamination = 0.05 has a consequence students always miss**
>
> It does not mean "5% of rows are anomalies". It means **the model will flag 5% of rows no matter what you give it**. Upload a perfectly clean file and it still flags 5%. The parameter defines where the cut-off sits, not how many anomalies exist.

The threshold problem (lines 62–88)

The original code used a hardcoded cut-off of `-0.0034`. That is wrong, and understanding why teaches something important about ML systems.

`decision_function` scores are only meaningful *relative to the forest that produced them*. A different dataset produces a different forest, which produces a different score distribution. A constant tuned on one file is a meaningless number on another.

`_resolve_threshold()` fixes this by taking the 5% quantile of the training scores — recalibrating for every file. Read the docstring on lines 67–81: it is honest that scikit-learn's `offset_` already does this internally, so the rule is equivalent to `model.predict() == -1`. The gain is portability, not accuracy.

The two entry points

`score_records()` (line 91) — used by the app. Fit, score, threshold, and optionally explain with SHAP. Note line 109: `anomaly_score` is the *negated* decision score, so that higher always means "more anomalous", which is more intuitive for display.

`run_model()` (line 155) — used by the CLI. This is the evaluation. Split, fit, calibrate, predict, then print four metrics for three different models.

SHAP attribution (lines 120–152)

The model says "this row is anomalous". SHAP says *why* — it assigns each of the seven features a number showing how much it pushed the score up or down. This is what turns a flag into an explanation, and it is the input the language model is later asked to put into words.

Notice the `try/except` wrapping the whole thing. If SHAP is not installed or fails, line 142 falls back to a simple statistical measure: how many standard deviations each feature sits from the training median, scaled by the anomaly score. The app keeps working, and `attribution_method` honestly reports which method was used.

> **Performance warning**
>
> SHAP is by far the slowest part of the system — roughly 170 ms per row on a 10,000-row file, which is about 28 minutes, or 99% of total runtime. Worse, per-row cost grows with the training set, because `max_samples=1.0` makes trees deeper on bigger files. The CLI evaluation does not run SHAP and is unaffected.

Why compare against a baseline? (lines 191–200)

A model scoring 0.75 F1 sounds good, but good *compared to what?* A reviewer will always ask: could a simple rule do this? So the code implements the obvious rule — "flag anything above the 90th percentile of cost" — and scores it on the same test set. The model gets 0.751; the rule gets 0.473.

Read the docstring: an earlier `mean + 2*std` rule scored exactly 0.0, and was replaced precisely because beating a rule that detects nothing proves nothing. **Always compare against the strongest simple alternative, not the weakest.**

7.2 Complete source code

```
1  from __future__ import annotations
2
3  import argparse
```

```python
from typing import List, Optional

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import IsolationForest
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from carci.shared import (
    FEATURE_COLUMNS,
    add_engineered_features,
    prepare_external_test_features,
    prepare_inference_features,
    prepare_train_test_features,
    print_split_summary,
)


# Retained only as a last-resort fallback. `decision_function` output is relative
# to whichever forest was fitted, so a constant tuned on one file carries no
# meaning on another upload; `_resolve_threshold` derives one per dataset instead.
DECISION_THRESHOLD = -0.0034
CONTAMINATION = 0.05
LABELED_MODEL_THRESHOLD = 0.36
ATTRIBUTION_METHOD = "SHAP TreeExplainer"

CATEGORICAL_CONTEXT_COLUMNS = [
    "service_name",
    "service",
    "region",
    "usage_type",
    "instance_type",
    "volume-type",
    "storage_type",
    "tag-project / tag-name",
    "tag_environment",
    "department",
    "environment",
    "project_name",
    "month",
]


def build_isolation_forest(contamination: float = CONTAMINATION) -> IsolationForest:
    return IsolationForest(
        n_estimators=1000,
        contamination=contamination,
        max_samples=1.0,
        max_features=1.0,
        random_state=123,
    )


def _resolve_threshold(
    model: IsolationForest,
    x_train: object,
    contamination: float = CONTAMINATION,
) -> float:
    """Derive a cut-off from this dataset's own score distribution.

```

```
    Scores from `decision_function` are only meaningful relative to the forest
    that produced them, so a fixed constant does not transfer between uploads.
    Taking the contamination-level quantile of the training scores recalibrates
    the cut-off for every file.

    Worth being precise about what this does and does not do: scikit-learn
    already sets `offset_` to that same quantile, so the value returned here is
    ~0.0 and the rule is equivalent to `model.predict() == -1`. The gain over the
    old hardcoded -0.0034 is therefore not a cleverer rule, it is that the
    cut-off tracks each dataset instead of being an arbitrary constant carried
    over from one file. It is kept as an explicit function so the operating point
    can later be moved away from `contamination` without re-tuning a magic number.
    """
    try:
        train_scores = np.asarray(model.decision_function(x_train), dtype=float)
        if train_scores.size == 0:
            return DECISION_THRESHOLD
        return float(np.quantile(train_scores, contamination))
    except Exception:
        return DECISION_THRESHOLD


def score_records(
    train_df: pd.DataFrame,
    score_df: Optional[pd.DataFrame] = None,
    decision_threshold: Optional[float] = None,
    explain: bool = True,
    contamination: float = CONTAMINATION,
) -> pd.DataFrame:
    """Score rows for anomaly. `decision_threshold=None` calibrates per dataset."""
    x_train, x_score, train_features, score_features = prepare_inference_features(train_df, score_df)

    model = build_isolation_forest(contamination)
    model.fit(x_train)

    if decision_threshold is None:
        decision_threshold = _resolve_threshold(model, x_train, contamination)

    result = score_features.copy()
    result["if_decision_score"] = model.decision_function(x_score)
    result["anomaly_score"] = -result["if_decision_score"]
    result["model_is_anomaly"] = (result["if_decision_score"] < decision_threshold).astype(int)
    result.attrs["attribution_method"] = "not calculated"
    result.attrs["decision_threshold"] = float(decision_threshold)

    if explain:
        result = _append_feature_attribution(result, model, x_score, train_features)

    return result


def _append_feature_attribution(
    result: pd.DataFrame,
    model: IsolationForest,
    x_score: object,
    train_features: pd.DataFrame,
) -> pd.DataFrame:
    try:
        import shap

        explainer = shap.TreeExplainer(model)
        shap_values = np.asarray(explainer.shap_values(x_score))
        # Some SHAP versions return (rows, features, outputs) for tree models;
        # collapse the trailing axis so the per-feature indexing below is valid.
        if shap_values.ndim == 3:
```

```
            shap_values = shap_values[:, :, 0]
        if shap_values.shape[1] != len(FEATURE_COLUMNS):
            raise ValueError(
                f"SHAP returned {shap_values.shape[1]} columns for "
                f"{len(FEATURE_COLUMNS)} features."
            )
        result.attrs["attribution_method"] = ATTRIBUTION_METHOD
    except Exception:
        train_center = np.asarray(train_features[FEATURE_COLUMNS].median())
        score_values = np.asarray(result[FEATURE_COLUMNS])
        train_scale = np.asarray(train_features[FEATURE_COLUMNS].std(ddof=0).replace(0, 1))
        shap_values = (score_values - train_center) / train_scale
        shap_values = shap_values * np.asarray(result["anomaly_score"]).reshape(-1, 1)
        result.attrs["attribution_method"] = "standardized feature impact fallback"

    for index, column in enumerate(FEATURE_COLUMNS):
        result[f"shap_{column}"] = shap_values[:, index]

    return result


def run_model(csv_file: str, test_csv_file: Optional[str] = None) -> None:
    df = pd.read_csv(csv_file)

    if test_csv_file:
        test_source_df = pd.read_csv(test_csv_file)
        x_train, x_test, y_true, train_df, test_df = prepare_external_test_features(
            df,
            test_source_df,
        )
    else:
        x_train, x_test, y_true, train_df, test_df = prepare_train_test_features(df)

    model = build_isolation_forest()
    model.fit(x_train)

    threshold = _resolve_threshold(model, x_train)
    scores = model.decision_function(x_test)
    y_pred = [1 if score < threshold else 0 for score in scores]
    baseline_pred = _cost_threshold_baseline(train_df, test_df)

    print("\nModel Evaluation")
    print_split_summary(train_df, test_df)
    print(f"Decision threshold (calibrated on this dataset): {threshold:.6f}")
    _print_metrics("Isolation Forest", y_true, y_pred)
    _print_metrics("Cost percentile baseline", y_true, baseline_pred)
    print(
        "\nNote: the baseline flags rows whose raw cost exceeds the 90th percentile of\n"
        "training spend. It scores poorly because these anomalies are unusual\n"
        "*combinations* of cost, usage, transfer and autoscaling rather than simply\n"
        "large amounts, which is the case for using a model over a single-column rule."
    )

    if not test_csv_file:
        _run_labeled_gradient_boosting_cv(df)


def _cost_threshold_baseline(train_df: pd.DataFrame, test_df: pd.DataFrame) -> List[int]:
    """Strongest simple cost rule, used as an honest comparison point.

    A `mean + 2*std` cut was previously used but flagged nothing at all on this
    data (F1 = 0.0), which made the comparison meaningless. The 90th-percentile
    cut is the best-performing variant of the same idea, so the model is measured
    against the rule at its strongest rather than a straw man.
    """
```

```
    threshold = float(train_df["cost"].quantile(0.90))
    return [1 if cost > threshold else 0 for cost in test_df["cost"]]


def _print_metrics(label: str, y_true: pd.Series, y_pred: List[int]) -> None:
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, zero_division=0)
    recall = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    print(f"\n{label}")
    print("Accuracy: ", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:   ", round(recall, 4))
    print("F1 Score: ", round(f1, 4))


def _run_labeled_gradient_boosting_cv(df: pd.DataFrame) -> None:
    prepared_df = add_engineered_features(df)
    if "is_anomaly" not in prepared_df.columns:
        return

    y = prepared_df["is_anomaly"].astype(int)
    anomaly_count = int((y == 1).sum())
    normal_count = int((y == 0).sum())
    if anomaly_count < 5 or normal_count < 5:
        print("\nLabeled Gradient Boosting")
        print("Skipped: at least 5 normal and 5 anomaly labels are required for 5-fold evaluation.")
        return

    x, preprocessor = _build_labeled_feature_matrix(prepared_df)
    model = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("classifier", GradientBoostingClassifier(random_state=123)),
        ]
    )
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    anomaly_probability = cross_val_predict(model, x, y, cv=cv, method="predict_proba")[:, 1]
    y_pred = (anomaly_probability >= LABELED_MODEL_THRESHOLD).astype(int)

    _print_metrics("Labeled Gradient Boosting (5-fold CV)", y, y_pred)
    print("Decision threshold:", LABELED_MODEL_THRESHOLD)
    print(
        "Caveat: this is a supervised model trained on is_anomaly labels, so it is not\n"
        "directly comparable to the unsupervised Isolation Forest above. The threshold\n"
        "was also chosen by inspecting these same folds, so the score is optimistic."
    )


def _build_labeled_feature_matrix(df: pd.DataFrame) -> tuple[pd.DataFrame, ColumnTransformer]:
    df = df.copy()
    numeric_columns = list(FEATURE_COLUMNS)
    for column in numeric_columns:
        df[column] = pd.to_numeric(df.get(column, 0), errors="coerce").fillna(0.0)
        log_column = f"log_{column}"
        df[log_column] = np.log1p(df[column].clip(lower=0))

    log_columns = [f"log_{column}" for column in numeric_columns]
    categorical_columns = [column for column in CATEGORICAL_CONTEXT_COLUMNS if column in df.columns]
    feature_columns = numeric_columns + log_columns + categorical_columns

    preprocessor = ColumnTransformer(
        transformers=[
            (
                "numeric",
```

```
                Pipeline(
                    steps=[
                        ("imputer", SimpleImputer(strategy="median")),
                        ("scaler", StandardScaler()),
                    ]
                ),
                numeric_columns + log_columns,
            ),
            (
                "categorical",
                Pipeline(
                    steps=[
                        ("imputer", SimpleImputer(strategy="constant", fill_value="missing")),
                        ("encoder", OneHotEncoder(handle_unknown="ignore", min_frequency=2)),
                    ]
                ),
                categorical_columns,
            ),
        ]
    )
    return df[feature_columns], preprocessor


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Run the Isolation Forest anomaly model.")
    parser.add_argument(
        "csv_file",
        nargs="?",
        default="dataset.csv",
        help="Path to the training CSV.",
    )
    parser.add_argument(
        "test_csv_file",
        nargs="?",
        default=None,
        help="Optional path to a separate testing CSV. All rows in this file are tested.",
    )
    args = parser.parse_args()
    run_model(args.csv_file, args.test_csv_file)
```

8 `carci/analysis.py` — From scores to explanations

`carci/analysis.py`

507 lines · Business rules and orchestration

8.1 Purpose and concepts

This is the largest file and the one that makes CARCI "Root Cause Intelligence" rather than just an anomaly detector. The model produces a number; this file turns it into a set of facts a finance manager can act on, and then hands those facts to `llm_reasoning.py` to be written up in English.

The pipeline: `build_analysis_result()` (line 466)

Read this function first. It orchestrates everything:

```
preprocess_cur_data()       clean the upload            (preprocessing.py)
        |
_score_input_rows()         run the model + SHAP        (isolation_forest_model.py)
        |
_attach_service_context()   compute per-service-day baselines
        |
_build_service_comparison() apply rules, build the records
        |                         |
        |                         +--> _apply_llm_explanations()   (llm_reasoning.py)
        |                              one Gemini call for the whole batch
        v
_build_daily_summary()      data for the trend chart
        |
_build_report_text()        the downloadable text report
```

The key idea: service-day aggregation (lines 102–138)

Read the docstring on lines 103–109 — it describes a real bug that was fixed. The original code compared each row against the average of all other rows. That is wrong, because a service which is *always* expensive would look anomalous on its priciest row even when nothing had changed.

The fix aggregates to the `(service, date)` level and compares each service-day against the mean of **that same service's other days**. Now the question becomes the right one: "*is AmazonEC2 more expensive today than AmazonEC2 usually is?*"

The detection rule (lines 147–150)

```
1  has_baseline  = service_day_count >= 2      # need another day to compare against
2  model_flagged = model_is_anomaly == 1       # the ML model said so
3  large_change  = percentage >= 75 AND impact >= $5   # the business rule said so
4
5  report if:  has_baseline AND (model_flagged OR large_change) AND impact > 0
```

Two independent detectors joined by OR. The model catches strange *combinations* that no rule would express; the rule catches large obvious increases the model might miss. The `$5` floor stops the system

reporting a rise from $0.02 to $0.05 — technically a 150% increase, but nobody cares.

The fallback (lines 153–159)

If nothing qualifies, rather than showing an empty screen the system displays the single highest-impact service-day marked `is_fallback = 1`. The UI labels it "Reference only" and it is **excluded from the anomaly count** (see line 475). This is good interface design: always show the user something, but never let informational content masquerade as a detection.

Deduplication (lines 164–166)

A service-day usually contains many rows. Without `drop_duplicates`, each one would produce its own identical card. The code sorts by anomaly score and keeps the highest-scoring row as the "driver" — one card per service-day, showing which specific line item caused it. Line 172 then caps the list at `MAX_ANOMALIES` (20), which is also the batch size the LLM is asked to handle.

Assembling the facts (lines 242–277)

`_build_root_cause_text()` assembles an English paragraph from parts: what happened, how it compares to baseline, where it came from, why the system flagged it, and what to do. Note the three-way branch at lines 243–254 — the wording changes depending on whether the model flagged it, the business rule flagged it, or it is only a reference row. The system never claims the model detected something when it did not.

Line 256 handles a subtle case: if a service has no previous spending day, the baseline is zero. Reporting "a 100% increase" would be misleading, so it says "new spend" instead.

`_recommendation_for()` (line 194) maps the usage type to concrete advice — data transfer issues get CDN advice, compute issues get instance-sizing advice. Simple keyword matching.

Two neighbours do the same job for the other card fields. `_headline_for()` (line 207) writes a one-line summary, and `_severity_for()` (line 221) ranks the anomaly by money at stake and then by how sharp the rise was. Both are computed from the pipeline's own figures, so a severity exists even with the LLM switched off — and note what the template headline cannot do: it can only describe the *symptom*, because naming a probable *cause* is precisely the judgement no keyword rule can make.

> **This paragraph is now the fallback, not the product**
>
> In the current design the text built here is the *deterministic template*. It is what the user sees when no API key is configured or the network is down. When the language model is available, its wording replaces this paragraph. Both are always computed and kept, so the two can be compared side by side.

Handing over to the LLM (lines 343–382)

`_apply_llm_explanations()` is the seam between the deterministic pipeline and the language model. It does three things, in this order:

1. **Preserve the template first (lines 356–363).** Every record copies its paragraph, advice and headline into `template_explanation`, `template_recommendation` and `template_headline`, and sets `explanation_method` to "deterministic template". This happens before any network call, so the deterministic text can never be

lost. `likely_causes` and `investigation_steps` are set to empty lists here, because they have no template equivalent at all.

2. **Call the model once (line 367).** All records go in a single batched request. See section 9.
3. **Overwrite only what came back (lines 371–382).** Results are matched by index. A record the model did not answer for keeps its template text, and each individual field falls back to the pipeline's own value, so a reply that omits the headline or the severity is degraded rather than blank.

Line 368 is the whole safety design in one statement: if `generate_explanations()` returns nothing, the function simply returns and the analysis proceeds unchanged. **The app has no hard dependency on the API being reachable.**

`explanation_method` is copied into the result metadata at lines 489–493, which is what the caption on page 3 of the app reads.

> **Teaching point — this is the real product**
>
> The Isolation Forest says: "*row 4,271 has a decision score of -0.0512.*"
>
> This file says: "*AmazonECS cost $1,049.81 on 16 January against a normal baseline of $201.57, an increase of 421%, driven by one resource in ap-southeast-1.*"
>
> Same information. Only one of them is useful to a human. Translating model output into decisions is where most of the value in an applied ML system lives.

8.2 Complete source code

```
1  from __future__ import annotations
2
3  from typing import Dict, List, Tuple
4
5  import numpy as np
6  import pandas as pd
7
8  from carci.isolation_forest_model import score_records
9  from carci.llm_reasoning import MODEL as LLM_MODEL
10 from carci.llm_reasoning import generate_explanations
11 from carci.preprocessing import preprocess_cur_data
12 from carci.shared import FEATURE_COLUMNS
13
14
15 BREAKDOWN_COLUMNS = ["service", "region", "resource", "tag", "usage_type"]
16
17 # Upper bound on anomaly cards rendered. The UI reports when this truncates.
18 MAX_ANOMALIES = 20
19
20
21 def _safe_pct_change(current_value: float, previous_value: float) -> float:
22     if previous_value <= 0 and current_value > 0:
23         return 100.0
24     if previous_value <= 0:
25         return 0.0
26     return ((current_value - previous_value) / previous_value) * 100.0
27
28
29 def _format_money(value: float) -> str:
```

```
    return f"${value:,.2f}"


def _to_breakdown(df: pd.DataFrame, column: str, limit: int = 8) -> List[Dict]:
    if df.empty or column not in df.columns:
        return []

    total_cost = float(df["cost"].sum())
    grouped = (
        df.groupby(column, dropna=False)["cost"]
        .sum()
        .sort_values(ascending=False)
        .head(limit)
        .reset_index()
        .rename(columns={column: "name"})
    )
    grouped["share_pct"] = np.where(total_cost > 0, (grouped["cost"] / total_cost) * 100.0, 0.0)
    grouped["cost"] = grouped["cost"].round(2)
    grouped["share_pct"] = grouped["share_pct"].round(2)
    return grouped.to_dict(orient="records")


def _top_name(breakdowns: Dict[str, List[Dict]], key: str, fallback: str) -> str:
    rows = breakdowns.get(key, [])
    if not rows:
        return fallback
    return str(rows[0]["name"])


def _top_share(breakdowns: Dict[str, List[Dict]], key: str) -> float:
    rows = breakdowns.get(key, [])
    if not rows:
        return 0.0
    return float(rows[0]["share_pct"])


def _build_daily_features(df: pd.DataFrame) -> pd.DataFrame:
    daily = df.groupby("date", as_index=False)["cost"].sum().sort_values("date").reset_index(drop=True)
    daily["previous_cost"] = daily["cost"].shift(1).fillna(0.0)
    daily["pct_change"] = [
        _safe_pct_change(curr, prev) for curr, prev in zip(daily["cost"], daily["previous_cost"])
    ]
    daily["rolling_mean_3"] = daily["cost"].rolling(window=3, min_periods=1).mean()
    daily["rolling_std_3"] = daily["cost"].rolling(window=3, min_periods=1).std(ddof=0).fillna(0.0)
    daily["cost_delta"] = daily["cost"] - daily["previous_cost"]

    mean_cost = float(daily["cost"].mean())
    std_cost = float(daily["cost"].std(ddof=0))
    daily["z_score"] = 0.0 if std_cost == 0 else (daily["cost"] - mean_cost) / std_cost
    return daily


def _score_input_rows(df: pd.DataFrame) -> Tuple[pd.DataFrame, str]:
    model_input = df.copy().sort_values(["date", "cost"], ascending=[True, False]).reset_index(drop=True)
    if len(model_input) < 2 or model_input["cost"].nunique() <= 1:
        model_input["if_decision_score"] = 0.0
        model_input["anomaly_score"] = 0.0
        model_input["model_is_anomaly"] = 0
        for column in FEATURE_COLUMNS:
            model_input[f"shap_{column}"] = 0.0
        return model_input, "not calculated"

    scored = score_records(model_input)
    attribution_method = scored.attrs.get("attribution_method", "not calculated")

```

```
    passthrough_columns = ["date", "service", "region", "resource", "tag", "usage_type"]
    for column in passthrough_columns:
        scored[column] = model_input[column].values

    return scored, attribution_method


def _attach_service_context(scored_df: pd.DataFrame) -> pd.DataFrame:
    """Attach a date-aware baseline to every scored row.

    The baseline answers "what does this service normally cost on a day?" by
    aggregating to the (service, date) level and averaging the service's *other*
    days. The previous row-level leave-one-out mean ignored dates entirely, so a
    service that was uniformly expensive looked anomalous on its priciest row
    even with no change over time.
    """
    scored_df = scored_df.copy()

    scored_df["service_date_cost"] = scored_df.groupby(["service", "date"])["cost"].transform("sum")

    per_day = scored_df.drop_duplicates(subset=["service", "date"])
    day_totals = per_day.groupby("service")["service_date_cost"].agg(["sum", "count"])
    scored_df["service_total_cost"] = scored_df["service"].map(day_totals["sum"])
    scored_df["service_day_count"] = scored_df["service"].map(day_totals["count"])

    scored_df["normal_value"] = np.where(
        scored_df["service_day_count"] > 1,
        (scored_df["service_total_cost"] - scored_df["service_date_cost"])
        / (scored_df["service_day_count"] - 1),
        0.0,
    )
    scored_df["impact_value"] = scored_df["service_date_cost"] - scored_df["normal_value"]
    # A zero baseline means first-ever spend, not a "100% increase".
    scored_df["is_new_spend"] = (scored_df["normal_value"] <= 0) & (scored_df["service_date_cost"] > 0)
    scored_df["percentage"] = [
        _safe_pct_change(curr, prev)
        for curr, prev in zip(scored_df["service_date_cost"], scored_df["normal_value"])
    ]
    scored_df["priority_score"] = (
        scored_df["model_is_anomaly"].astype(float) * 2.0
        + scored_df["anomaly_score"].clip(lower=0)
        + scored_df["impact_value"].clip(lower=0) / max(float(scored_df["service_date_cost"].max()), 1.0)
    )
    return scored_df


def _candidate_anomaly_rows(scored_df: pd.DataFrame) -> Tuple[pd.DataFrame, int]:
    """Select anomalous service-days and the row that drove each one.

    Returns the capped candidates plus the pre-cap count so the caller can tell
    the user when the list was truncated.
    """
    has_baseline = scored_df["service_day_count"] >= 2
    model_flagged = scored_df["model_is_anomaly"] == 1
    large_change = (scored_df["percentage"] >= 75.0) & (scored_df["impact_value"] >= 5.0)
    candidates = scored_df[has_baseline & (model_flagged | large_change) & (scored_df["impact_value"] >
0)].copy()
    candidates["is_fallback"] = 0

    if candidates.empty and not scored_df.empty:
        top_row = scored_df[has_baseline].sort_values(
            ["impact_value", "anomaly_score", "cost"],
            ascending=[False, False, False],
        ).head(1).copy()
        top_row["is_fallback"] = 1
```

```
        candidates = top_row

    # One card per service-day, keeping the highest-scoring row as the driver.
    # Without this, several rows of the same service and date produced duplicate
    # cards carrying identical breakdowns.
    candidates = candidates.sort_values(
        ["anomaly_score", "cost"], ascending=[False, False]
    ).drop_duplicates(subset=["service", "date"], keep="first")

    total_found = int(len(candidates))
    candidates = candidates.sort_values(
        ["priority_score", "impact_value", "cost"],
        ascending=[False, False, False],
    ).head(MAX_ANOMALIES)
    return candidates, total_found


def _get_feature_contributions(row: pd.Series) -> Tuple[List[Dict], List[str]]:
    feature_rows = []
    for column in FEATURE_COLUMNS:
        shap_value = float(row.get(f"shap_{column}", 0.0))
        feature_rows.append(
            {
                "feature": column,
                "feature_value": round(float(row.get(column, 0.0)), 4),
                "shap_value": round(shap_value, 4),
                "impact_strength": round(abs(shap_value), 4),
            }
        )

    feature_rows.sort(key=lambda item: item["impact_strength"], reverse=True)
    top_features = [item["feature"] for item in feature_rows[:3] if item["impact_strength"] > 0]
    return feature_rows, top_features


def _recommendation_for(record: Dict) -> str:
    usage_type = record["usage_type"].lower()
    if "datatransfer" in usage_type or "bandwidth" in usage_type:
        return "Review outbound data transfer, cross-region traffic, and CDN or caching options."
    if "boxusage" in usage_type or "instance" in usage_type or "compute" in usage_type:
        return "Review instance size, running hours, scaling activity, and idle compute resources."
    if "storage" in usage_type or "volume" in usage_type or "glacier" in usage_type:
        return "Review storage tier, lifecycle policies, and unexpected retained data volume."
    if "request" in usage_type:
        return "Review request volume, retry behavior, and application traffic sources."
    return "Review the listed service, resource, tag, and usage type before increasing the budget."


def _headline_for(record: Dict) -> str:
    """A one-line summary for the card header.

    The template can only describe the symptom, because naming a cause is exactly
    the judgement it cannot make. The LLM replaces this with a suspected cause.
    """
    if record["is_new_spend"]:
        return f"New {record['usage_type']} spend on {record['service_name']}"
    return (
        f"{record['service_name']} {record['usage_type']} up "
        f"{record['percentage']:.0f}% in {record['region']}"
    )


def _severity_for(record: Dict) -> str:
    """Rank an anomaly by money at stake, then by how sharp the rise was.

```

```
    Computed from the pipeline's own figures so that a severity exists even with
    the LLM switched off. When the model runs it may override this with its own
    reading, which also weighs the usage type and the feature pattern.
    """
    if record["is_fallback"]:
        # Nothing crossed the threshold; this row is context, not an incident.
        return "low"
    impact = record["impact_value"]
    percentage = record["percentage"]
    if impact >= 1000 or (impact >= 250 and percentage >= 500):
        return "critical"
    if impact >= 250 or (impact >= 50 and percentage >= 200):
        return "high"
    if impact >= 50:
        return "medium"
    return "low"


def _build_root_cause_text(record: Dict) -> str:
    if record["is_fallback"]:
        flag_text = (
            "No row crossed the anomaly threshold, so this service-day is shown only because it "
            "carries the largest positive cost impact. Treat it as informational, not as a detection."
        )
    elif record["model_flag"]:
        flag_text = "The Isolation Forest model flagged the driving line item as anomalous."
    else:
        flag_text = (
            "The model did not flag this service-day, but the cost increase exceeded the "
            "review thresholds (75% above baseline and at least $5 of impact)."
        )

    if record["is_new_spend"]:
        comparison_text = (
            f"The observed cost was {_format_money(record['anomaly_value'])}. This service has no prior "
            f"spending day in the uploaded file, so no baseline comparison is possible and this is "
            f"treated as new spend."
        )
    else:
        comparison_text = (
            f"The observed cost was {_format_money(record['anomaly_value'])}, compared with a normal
daily "
            f"baseline of {_format_money(record['normal_value'])} for this service. This is an increase
of "
            f"approximately {record['percentage']:.0f}%."
        )

    top_features = ", ".join(record["top_ml_features"]) if record["top_ml_features"] else "cost behavior"
    return (
        f"A cost anomaly was detected for {record['service_name']} on {record['display_date']}.\n\n"
        f"{comparison_text}\n\n"
        f"The main region was {record['region']} ({record['region_share_pct']:.0f}% of this service/date
cost). "
        f"The main resource was {record['resource']}, with usage type {record['usage_type']} and tag
{record['tag']}.\n\n"
        f"{flag_text} {record['attribution_method']} identified the strongest feature impacts from
{top_features}.\n\n"
        f"Recommended action: {record['recommendation']}"
    )


def _build_anomaly_record(row: pd.Series, df: pd.DataFrame, attribution_method: str) -> Dict:
    date_value = pd.Timestamp(row["date"])
    same_service_date = df[(df["date"] == row["date"]) & (df["service"] == row["service"])]
    same_date = df[df["date"] == row["date"]]
```

```
    breakdowns = {
        "service": _to_breakdown(same_date, "service"),
        "region": _to_breakdown(same_service_date, "region"),
        "resource": _to_breakdown(same_service_date, "resource"),
        "tag": _to_breakdown(same_service_date, "tag"),
        "usage_type": _to_breakdown(same_service_date, "usage_type"),
    }
    feature_contributions, top_features = _get_feature_contributions(row)

    record = {
        "service_name": str(row["service"]),
        "date": date_value.strftime("%Y-%m-%d"),
        "display_date": date_value.strftime("%d %B %Y"),
        "region": _top_name(breakdowns, "region", str(row["region"])),
        "resource": _top_name(breakdowns, "resource", str(row["resource"])),
        "tag": _top_name(breakdowns, "tag", str(row["tag"])),
        "usage_type": _top_name(breakdowns, "usage_type", str(row["usage_type"])),
        # Day-level totals for the service, with the driving line item's cost kept
        # alongside so the card shows both the "what" and the "which row".
        "anomaly_value": round(float(row["service_date_cost"]), 2),
        "driver_row_cost": round(float(row["cost"]), 2),
        "normal_value": round(float(row["normal_value"]), 2),
        "percentage": round(float(row["percentage"]), 2),
        "impact_value": round(float(row["impact_value"]), 2),
        "model_anomaly_score": round(float(row.get("anomaly_score", 0.0)), 6),
        "model_decision_score": round(float(row.get("if_decision_score", 0.0)), 6),
        "model_flag": int(row.get("model_is_anomaly", 0)),
        "is_fallback": int(row.get("is_fallback", 0)),
        "is_new_spend": bool(row.get("is_new_spend", False)),
        "region_share_pct": _top_share(breakdowns, "region"),
        "resource_share_pct": _top_share(breakdowns, "resource"),
        "tag_share_pct": _top_share(breakdowns, "tag"),
        "usage_type_share_pct": _top_share(breakdowns, "usage_type"),
        "breakdowns": breakdowns,
        "feature_contributions": feature_contributions,
        "top_ml_features": top_features,
        "attribution_method": attribution_method,
    }
    record["recommendation"] = _recommendation_for(record)
    record["severity"] = _severity_for(record)
    record["headline"] = _headline_for(record)
    record["root_cause_analysis"] = _build_root_cause_text(record)
    return record


def _build_service_comparison(
    scored_df: pd.DataFrame, prepared_df: pd.DataFrame, attribution_method: str
) -> Tuple[List[Dict], int]:
    candidates, total_found = _candidate_anomaly_rows(scored_df)
    records = [_build_anomaly_record(row, prepared_df, attribution_method) for _, row in
candidates.iterrows()]
    ordered = sorted(
        records,
        key=lambda item: (item["model_flag"], item["impact_value"], item["anomaly_value"]),
        reverse=True,
    )
    _apply_llm_explanations(ordered)
    return ordered, total_found


def _apply_llm_explanations(records: List[Dict]) -> None:
    """Replace the template narrative with the LLM's full investigation.

    The LLM is the primary explanation source. It supplies the narrative, the
    headline, the severity, the ranked cause hypotheses and the investigation
```

```
    steps; the last two have no template equivalent and stay empty without it.

    Every record still keeps `template_explanation`, `template_recommendation`
    and `template_headline`, so the deterministic text is never lost and the two
    can be compared. `explanation_method` records which one the card is actually
    showing. When the layer is off or the API call fails, nothing changes beyond
    those bookkeeping fields, so the analysis never depends on the network.
    """
    for record in records:
        record["template_explanation"] = record["root_cause_analysis"]
        record["template_recommendation"] = record["recommendation"]
        record["template_headline"] = record["headline"]
        record["explanation_method"] = "deterministic template"
        # Hypotheses and investigation steps have no template equivalent: naming a
        # probable cause is a judgement no keyword rule can make. In template mode
        # these stay empty and the UI simply omits those sections.
        record["likely_causes"] = []
        record["investigation_steps"] = []

    generated = generate_explanations(records)
    if not generated:
        return

    for index, item in generated.items():
        if not 0 <= index < len(records):
            continue
        records[index]["root_cause_analysis"] = item["explanation"]
        records[index]["explanation_method"] = f"LLM ({LLM_MODEL})"
        records[index]["likely_causes"] = item["likely_causes"]
        records[index]["investigation_steps"] = item["investigation_steps"]
        # Each remaining field falls back to the pipeline's own value, so a reply
        # that omits one is degraded rather than blank.
        if item.get("recommended_action"):
            records[index]["recommendation"] = item["recommended_action"]
        if item.get("headline"):
            records[index]["headline"] = item["headline"]
        if item.get("severity"):
            records[index]["severity"] = item["severity"]


def _build_daily_summary(df: pd.DataFrame, anomaly_records: List[Dict]) -> List[Dict]:
    daily = _build_daily_features(df)
    anomaly_dates = {record["date"] for record in anomaly_records}
    daily["has_anomaly"] = daily["date"].dt.strftime("%Y-%m-%d").isin(anomaly_dates)
    daily["date"] = daily["date"].dt.strftime("%Y-%m-%d")
    numeric_columns = ["cost", "previous_cost", "pct_change", "rolling_mean_3", "cost_delta", "z_score"]
    daily[numeric_columns] = daily[numeric_columns].round(4)
    return daily.to_dict(orient="records")


def _build_report_text(result: Dict) -> str:
    lines = [
        "CARCI Cost Anomaly Root Cause Report",
        "",
        f"Rows analyzed: {result['metadata']['row_count']}",
        f"Date range: {result['metadata']['date_range']}",
        f"Total cost: {_format_money(result['metadata']['total_cost'])}",
        f"Detected anomalies: {result['metadata']['anomaly_count']}",
        f"Attribution method: {result['metadata']['attribution_method']}",
        "",
    ]

    if result["metadata"]["truncated"]:
        lines.append(
            f"Note: {result['metadata']['total_found']} service-days qualified; "
```

```
            f"the {result['metadata']['shown_count']} highest-priority are listed below."
        )
        lines.append("")

    if not result["service_comparison"]:
        lines.append("No strong anomalies were detected in the uploaded file.")
        return "\n".join(lines)

    if result["metadata"]["anomaly_count"] == 0:
        lines.append(
            "No service-day crossed the anomaly threshold. The entry below is the "
            "highest-impact service-day, shown for reference only."
        )
        lines.append("")

    for index, record in enumerate(result["service_comparison"], start=1):
        increase = (
            "new spend (no prior baseline)"
            if record["is_new_spend"]
            else f"{record['percentage']:.0f}%"
        )
        lines.extend(
            [
                f"{index}. [{record['severity'].upper()}] {record['service_name']} - {record['date']}"
                + ("  [reference only]" if record["is_fallback"] else ""),
                f"   {record['headline']}",
                "",
                f"   Cost: {_format_money(record['anomaly_value'])}",
                f"   Normal baseline: {_format_money(record['normal_value'])}",
                f"   Increase: {increase}",
                f"   Region: {record['region']}",
                f"   Resource: {record['resource']}",
                f"   Tag: {record['tag']}",
                f"   Usage type: {record['usage_type']}",
                f"   Top ML features: {', '.join(record['top_ml_features']) or 'cost behavior'}",
            ]
        )

        if record["likely_causes"]:
            lines.append("   Likely causes:")
            for rank, cause in enumerate(record["likely_causes"], start=1):
                lines.append(f"     {rank}. [{cause['confidence']}] {cause['cause']}")
                lines.append(f"        Evidence: {cause['evidence']}")

        if record["investigation_steps"]:
            lines.append("   Investigation steps:")
            for step_number, step in enumerate(record["investigation_steps"], start=1):
                lines.append(f"     {step_number}. {step}")

        lines.extend([f"   Recommendation: {record['recommendation']}", ""])
    return "\n".join(lines)


def build_analysis_result(raw_df: pd.DataFrame) -> Dict:
    prepared_df = preprocess_cur_data(raw_df)
    scored_df, attribution_method = _score_input_rows(prepared_df)
    scored_df = _attach_service_context(scored_df)

    service_comparison, total_found = _build_service_comparison(
        scored_df, prepared_df, attribution_method
    )
    # A fallback row is a "nothing crossed the threshold" placeholder, not a detection.
    confirmed_count = sum(1 for record in service_comparison if not record["is_fallback"])
    date_min = prepared_df["date"].min().strftime("%Y-%m-%d")
    date_max = prepared_df["date"].max().strftime("%Y-%m-%d")
```

```
    result = {
        "metadata": {
            "row_count": int(len(prepared_df)),
            "service_count": int(prepared_df["service"].nunique()),
            "anomaly_count": confirmed_count,
            "shown_count": int(len(service_comparison)),
            "total_found": total_found,
            "truncated": bool(total_found > len(service_comparison)),
            "total_cost": round(float(prepared_df["cost"].sum()), 2),
            "date_range": f"{date_min} to {date_max}",
            "attribution_method": attribution_method,
            "explanation_method": (
                service_comparison[0]["explanation_method"]
                if service_comparison
                else "deterministic template"
            ),
        },
        "daily_summary": _build_daily_summary(prepared_df, service_comparison),
        "overall_breakdowns": {column: _to_breakdown(prepared_df, column) for column in
BREAKDOWN_COLUMNS},
        "service_comparison": service_comparison,
        "service_root_causes": [
            {
                "service_name": row["service_name"],
                "root_cause_analysis": row["root_cause_analysis"],
            }
            for row in service_comparison
        ],
    }
    result["report_text"] = _build_report_text(result)
    return result
```

9 `carci/llm_reasoning.py` — Writing the root cause

`carci/llm_reasoning.py`

439 lines · The language model layer

9.1 Purpose and concepts

This file writes the investigation the user actually reads. Everything before it in the pipeline produces numbers; this is the step that produces reasoning.

It sends the anomaly facts computed by `analysis.py` to Google Gemini (`gemini-3.5-flash-lite`, line 43) over the REST API, and receives back a structured report per anomaly: a headline naming the suspected cause, a severity, a narrative explanation, ranked cause hypotheses with confidence and evidence, the checks that would confirm them, and a recommended action.

> **The idea that makes this safe: facts and hypotheses are different things**
>
> A root cause report has to do two incompatible things. It must quote figures with absolute fidelity, and it must speculate about causes — because the cause is never in the billing data. Treating those as one task is what makes language models dangerous in financial reporting.
>
> So the prompt splits them explicitly. **Facts** may only be repeated from the input; the model never calculates, re-rounds or invents a figure. **Hypotheses** are where the model is expected to reason past the data, but each one must be ranked, carry a confidence, cite the field it rests on, and be worded as a possibility rather than a finding.
>
> The result reads like an analyst wrote it, without the model ever being trusted with arithmetic.

Where the key comes from

`api_key()` (line 198) checks, in order, the `GEMINI_API_KEY` and `GOOGLE_API_KEY` environment variables, falling back to a `.env` file at the project root read by `_load_env_file()` (line 175). That loader is deliberately about twenty lines of plain Python rather than a dependency on `python-dotenv`, and it never overwrites a variable that is already set.

```
# .env at the project root - never commit this file
GEMINI_API_KEY=your-key-here

# Set to 0 to force the deterministic template instead of the LLM.
CARCI_LLM_REASONING=1
```

`is_enabled()` (line 207) makes the feature **on by default whenever a key exists**. Setting `CARCI_LLM_REASONING=0` forces the deterministic template instead, which is how you demonstrate the fallback path without deleting your key.

What is sent

`_to_payload()` (line 219) reduces each anomaly record to the facts the model is allowed to talk about. This narrowing is a security and correctness decision, not a bandwidth one. Read the docstring on lines 220–226: passing the whole record would hand the model fields it has no business repeating, and *every extra key is a chance for it to state something the pipeline never computed*.

Note also what is deliberately *absent*: the rule-based recommendation from `_recommendation_for()` is not included, so the action the model returns is its own reading of the facts rather than a rewrite of a keyword rule.

One entry of the JSON array, taken from a real run:


```
{
  "index": 0,
  "service": "AmazonEC2",
  "date": "13 January 2026",
  "observed_cost_usd": 2979.72,
  "baseline_cost_usd": 446.67,
  "increase_pct": 567.1,
  "impact_usd": 2533.05,
  "driver_line_item_cost_usd": 1011.65,
  "region": "ap-southeast-1",
  "region_share_pct": 72.01,
  "resource": "db-1cd8dd14-73b0-43ee-bcab-4831009223e3",
  "usage_type": "CryptoMiningTraffic",
  "tag": "AIResearch",
  "top_features": ["usage_quantity", "cost", "data_transfer_gb"],
  "feature_attribution": [
    {"feature": "usage_quantity",   "value": 16049.04, "shap_value": -2.3399},
    {"feature": "cost",             "value": 1011.65,  "shap_value": -2.2855},
    {"feature": "data_transfer_gb", "value": 770.18,   "shap_value": -2.2010}
  ],
  "attribution_method": "SHAP TreeExplainer",
  "model_flagged": true,
  "reference_only": false,
  "no_prior_baseline": false
}
```


The system prompt (lines 60–127)

The prompt is in three parts: the FACTS rules, the HYPOTHESES rules, and a specification of each field. Every rule exists to prevent a specific failure:

Rule	The failure it prevents
Facts: use only the numbers present in the input	The model inventing, re-rounding or recomputing a figure — the single most damaging error in a billing report
Facts: do not claim a detection that did not happen	Presenting a `reference_only` row, or one the model did not flag, as a confirmed detection
Facts: new spend is not an increase	Describing a first-ever charge as a percentage rise against a zero baseline
Hypotheses: ground every cause in a given field	Plausible-sounding speculation with nothing behind it
Hypotheses: rank them and state confidence	Three guesses presented as though equally likely
Hypotheses: never word a guess as a finding	A reader acting on "was caused by" when the evidence only supports "consistent with"
Fields: headline names the cause, not the symptom	Sixteen cards that all read "service X cost increase"
Fields: a specific mechanism, not a category	"Increased usage", which is a restatement of the anomaly rather than an explanation
Fields: three or four paragraphs, no markdown	Headings and bullets rendering as literal asterisks inside a Streamlit expander

Forcing a machine-readable reply (lines 132–173, 293–300)

The request sets `responseMimeType: application/json` together with an explicit `responseSchema`. The schema requires every field above, and constrains `severity` and `confidence` to enumerations, so the UI can colour a badge from the value without guarding against a word it has no palette for.

This matters more than it looks. Without it, the reply is prose that has to be scraped with regular expressions, and every change in the model's phrasing breaks the parser. With it, the reply is parsed by `json.loads()` and the structure is guaranteed by the API rather than hoped for.

`temperature` is set to 0.2 (line 298). Even for hypothesis generation this stays low: the creativity wanted here is domain reasoning, not linguistic variety, and drift at higher temperatures reads as invented numbers.

`MAX_OUTPUT_TOKENS` (line 47) is 32768. The single paragraph this layer originally returned fit comfortably in 8192; a full investigation for twenty anomalies runs to roughly 8,000 output tokens on its own, and a truncated reply loses whole records.

What comes back

```
{
  "index": 0,
  "headline": "Crypto-mining traffic on EC2 instance db-1cd8dd14",
  "severity": "critical",
  "explanation": "On 13 January 2026, AmazonEC2 spend surged by 567.10% to reach an
                  observed cost of 2979.72 USD ... (three more paragraphs)",
  "likely_causes": [
    {"cause":      "Compromised EC2 instance running unauthorized mining scripts",
     "confidence": "high",
     "evidence":   "usage_type is CryptoMiningTraffic with 16049.04 usage quantity
                    and 770.18 GB data transfer"},
    {"cause":      "Unintended deployment of heavy proof-of-work algorithms under
                    the AIResearch tag",
     "confidence": "medium",
     "evidence":   "Tag is AIResearch and ap-southeast-1 holds 72.01% region share"}
  ],
  "investigation_steps": [
    "Open the Amazon EC2 Console and inspect resource db-1cd8dd14-... in
     ap-southeast-1.",
    "Review VPC Flow Logs and CloudTrail for unexpected inbound connections or
     credential misuse on the instance.",
    "Check security group rules associated with the resource for overly
     permissive access."
  ],
  "recommended_action": "Immediately isolate and stop the compromised instance
                         db-1cd8dd14-..., revoke any compromised IAM credentials,
                         and perform a security audit of the AIResearch
                         environment."
}
```

`generate_explanations()` (line 367) parses this into a dictionary keyed by `index`, which is how each investigation finds its way back to the anomaly it describes.

Trusting the reply as little as possible (lines 324–364)

The schema guarantees the *shape* of the reply, not its *content*. Three small cleaners sit between the API and the UI:

- `_clean_choice()` (line 324) drops any severity or confidence that is not one of the expected words, rather than passing an unknown string to a UI that colours a badge from it.
- `_clean_causes()` (line 335) discards any hypothesis missing either its mechanism or its evidence. A cause with no evidence is exactly the unsupported speculation the prompt forbids, so it never reaches the screen.
- `_clean_steps()` (line 361) strips blanks and caps the list.

A record whose `explanation` came back empty is skipped entirely, so it keeps its template text instead of rendering as a half-filled card.

> **Teaching point — validate at the boundary**
>
> The schema is enforced by someone else's server. These cleaners are the first code you control that touches the reply, and they are where the assumption "the response is well-formed" gets checked instead of trusted. Treat any external API the same way, including one whose contract you believe you have pinned down.

Batching (line 54)

`MAX_RECORDS_PER_CALL = 20`, and all twenty anomalies travel in a single request. The comment states the reason plainly: twenty separate requests would cost twenty times the latency for the same work. This also matches `MAX_ANOMALIES` in `analysis.py`, so a full page of results is always one API call.

Failure handling — read this part twice

`_call_gemini()` (line 269) retries once, and only for the transient status codes 429, 500 and 503 (line 309). A 400 or a 403 means the request or the key is wrong; retrying those just doubles the wait before the same failure.

Lines 405–409 handle a case that is easy to miss: a safety block returns HTTP 200 with no content parts at all. The code checks `finishReason` *before* touching `parts[0]`, so a blocked response produces a readable diagnostic instead of an `IndexError`. `MAX_TOKENS` is treated as recoverable, because a truncated reply may still contain usable entries.

Every failure path sets `_LAST_ERROR` (line 58) and returns `None`. Read the docstring on lines 375–377:

"None is not an error condition. It is the normal signal that the caller should keep the deterministic template text."

> **Teaching point — graceful degradation**
>
> There are five separate ways this layer can produce nothing: no key, the feature switched off, a network failure, an HTTP error, and an unparseable reply. All five lead to the same outcome — the user gets the template paragraph and the app keeps working. None of them raises.
>
> The counterpart is that a silent fallback is invisible, which is why `last_error()` (line 215) exists and why the app prints which method wrote the text. **Degrade quietly, but always leave a way to find out that you did.**
>
> Note what the fallback costs. The narrative, headline and severity all have template equivalents, so they survive. The ranked hypotheses and investigation steps do not — naming a probable cause is a judgement no keyword rule can make. Offline, those sections are simply absent from the card rather than filled with something weaker.

How to check it is actually working

Because the fallback is silent by design, "the explanations look fine" is not evidence that the model ran. Three checks, in increasing order of certainty:

1. **The caption on page 3 of the app.** It reads either "*Explanations written by LLM (gemini-3.5-flash-lite)*" or "*Explanations from the deterministic template*".
2. **The Recommended action line.** It is only rendered as a separate bold line in LLM mode. In template mode the advice sits inside the paragraph.
3. **Inspect the result directly.** `result["metadata"]["explanation_method"]` names the source, and `llm_reasoning.last_error()` explains any failure.

> **Cost and privacy — worth stating in a report**
>
> The billing figures for up to twenty anomalies leave the machine and are sent to a third-party API. A measured batch of sixteen anomalies costs roughly 7,700 input tokens and 8,000 output tokens. No raw CSV is uploaded and no credentials travel with it, but resource identifiers and tag names do. For coursework this is fine; for a real finance deployment it is a decision that needs to be made deliberately, and the `CARCI_LLM_REASONING=0` switch exists so it can be turned off.

9.2 Complete source code

```
"""LLM layer that writes the root-cause investigation for each anomaly.

This replaces the deterministic template as the *primary* explanation source. The
model is given a JSON array of anomalies whose every figure has already been
computed by `analysis.py` from the billing data, and it returns a structured
investigation for each one: a headline naming the suspected cause, a severity, a
narrative explanation, ranked cause hypotheses with confidence and supporting
evidence, the checks that would confirm or eliminate them, and a recommended
action.

The split that makes this safe is between facts and hypotheses. Figures, names,
dates and features are quoted from the input and never recomputed - the pipeline
owns those. Causes are the model's own reasoning, are ranked rather than asserted,
and each one has to cite the field it rests on. So the report reads like an
analyst wrote it without the model ever being trusted with arithmetic.

Provider: Google Gemini (`gemini-3.5-flash-lite`) over the REST API, called with
`responseMimeType=application/json` and an explicit `responseSchema`, so the
reply is machine-parseable rather than prose that has to be scraped.

Credentials, in priority order:

    GEMINI_API_KEY / GOOGLE_API_KEY environment variable
    GEMINI_API_KEY=... line in a `.env` file at the project root

Enabled by default whenever a key is present. Set CARCI_LLM_REASONING=0 to force
the deterministic template instead. If the key is missing, the network call
fails, or the reply cannot be parsed, the caller silently keeps the template
text, so the app still runs offline and never breaks on an API outage.
"""

from __future__ import annotations

import json
import os
import time
import urllib.error
import urllib.parse
import urllib.request
```

```
from pathlib import Path
from typing import Dict, List, Optional

MODEL = "gemini-3.5-flash-lite"
API_ROOT = "https://generativelanguage.googleapis.com/v1beta/models"
# A hypothesis-driven report runs several times longer than the single paragraph
# this layer used to return, and a truncated reply loses whole records.
MAX_OUTPUT_TOKENS = 32768
TIMEOUT_SECONDS = 90
ENABLE_FLAG = "CARCI_LLM_REASONING"
KEY_VARIABLES = ("GEMINI_API_KEY", "GOOGLE_API_KEY")

# Batched into one request rather than one call per anomaly: 20 separate requests
# would cost 20x the latency for the same work.
MAX_RECORDS_PER_CALL = 20

# Surfaced by the UI when the layer is enabled but produced nothing, so a failed
# call is diagnosable instead of silently looking like "the feature is off".
_LAST_ERROR: Optional[str] = None

SYSTEM_PROMPT = """You are a senior AWS cost investigator writing the root-cause
section of a billing incident report.

You will receive a JSON array of detected cost anomalies. Every figure in each
entry has ALREADY been computed from the customer's billing data by a statistical
pipeline and a SHAP feature-attribution step.

Your output contains two different kinds of statement, and the rules differ.

FACTS - the numbers, names, dates, regions, resources and features you are given.

  * Use ONLY what is in the input. Never calculate, re-round, estimate or invent
    a figure. If a value is not in the input, do not state it.
  * Do not claim the model detected something when `model_flagged` is false, and
    never present a `reference_only` entry as a confirmed detection.
  * When `no_prior_baseline` is true there is no comparison to draw. Call it new
    spend; do not describe it as an increase.

HYPOTHESES - what plausibly caused the anomaly.

  * Here you are expected to reason past the data and propose specific engineering
    explanations. This is the part of the report a human analyst would write.
  * Ground every hypothesis in something you were given: the usage type, the
    resource, how concentrated the region share is, the tag, and which features
    the attribution step found most important.
  * Rank them, and say how confident you are. A hypothesis you cannot tie to
    anything in the input is not worth listing.
  * Never state a hypothesis as established fact. "Consistent with", "suggests"
    and "would explain" are correct. "Was caused by" is not.

Fields to return:

`headline` - under 70 characters. Name the suspected cause, not the symptom.
  Weak: "AmazonEC2 cost increase". Strong: "Crypto-mining traffic on an untagged
  research instance".

`severity` - one of critical, high, medium, low. Judge it from `impact_usd` and
  `increase_pct` together with how alarming the usage type and feature pattern
  are. A `reference_only` entry is never above low.

`explanation` - three or four short paragraphs of plain prose. No headings, no
  bullet points, no markdown.
    Paragraph 1: what changed, in money and in time.
    Paragraph 2: where the cost concentrated - region, resource, usage type, tag -
      and what that concentration implies.
```

```
    Paragraph 3: what the feature attribution says drove the score, in plain
      language rather than raw column names.
    Paragraph 4, only if it earns its place: what makes this case unusual, or why
      it should be escalated.
  Write for a finance manager who is technical enough to act but did not build
  the system.

`likely_causes` - two to four ranked hypotheses, most likely first. Each carries:
    `cause` - a specific mechanism, not a category. "An autoscaling group scaled
      out under load and never scaled back in" beats "increased usage".
    `confidence` - high, medium or low.
    `evidence` - the specific figure or field from the input that supports it.

`investigation_steps` - two to four concrete checks in the order you would run
  them. Name the AWS console page, CLI command or log to open, and the actual
  resource, region or tag to filter on. Each step should confirm or eliminate one
  of the hypotheses above.

`recommended_action` - one or two sentences on what to do once the cause is
  confirmed. Specific to this anomaly, never generic advice that would fit any of
  them.

Return one entry per input entry, matched by `index`."""

CONFIDENCE_LEVELS = ("high", "medium", "low")
SEVERITY_LEVELS = ("critical", "high", "medium", "low")

RESPONSE_SCHEMA = {
    "type": "OBJECT",
    "properties": {
        "explanations": {
            "type": "ARRAY",
            "items": {
                "type": "OBJECT",
                "properties": {
                    "index": {"type": "INTEGER"},
                    "headline": {"type": "STRING"},
                    "severity": {"type": "STRING", "enum": list(SEVERITY_LEVELS)},
                    "explanation": {"type": "STRING"},
                    "likely_causes": {
                        "type": "ARRAY",
                        "items": {
                            "type": "OBJECT",
                            "properties": {
                                "cause": {"type": "STRING"},
                                "confidence": {"type": "STRING", "enum": list(CONFIDENCE_LEVELS)},
                                "evidence": {"type": "STRING"},
                            },
                            "required": ["cause", "confidence", "evidence"],
                        },
                    },
                    "investigation_steps": {"type": "ARRAY", "items": {"type": "STRING"}},
                    "recommended_action": {"type": "STRING"},
                },
                "required": [
                    "index",
                    "headline",
                    "severity",
                    "explanation",
                    "likely_causes",
                    "investigation_steps",
                    "recommended_action",
                ],
            },
        }
```

```
    },
    "required": ["explanations"],
}


def _load_env_file() -> None:
    """Read `.env` at the project root into os.environ, without overwriting it.

    Keeps the key out of the source tree. Deliberately minimal so the project
    does not need python-dotenv as a dependency.
    """
    env_path = Path(__file__).resolve().parent.parent / ".env"
    if not env_path.is_file():
        return
    try:
        for line in env_path.read_text(encoding="utf-8").splitlines():
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            name, _, value = line.partition("=")
            name = name.strip()
            value = value.strip().strip('"').strip("'")
            if name and name not in os.environ:
                os.environ[name] = value
    except OSError:
        return


def api_key() -> str:
    _load_env_file()
    for variable in KEY_VARIABLES:
        value = os.environ.get(variable, "").strip()
        if value:
            return value
    return ""


def is_enabled() -> bool:
    """On by default when a key exists; CARCI_LLM_REASONING=0 forces it off."""
    flag = os.environ.get(ENABLE_FLAG, "").strip().lower()
    if flag in {"0", "false", "no", "off"}:
        return False
    return bool(api_key())


def last_error() -> Optional[str]:
    return _LAST_ERROR


def _to_payload(record: Dict, index: int) -> Dict:
    """Reduce an anomaly record to the facts the model is allowed to talk about.

    Deliberately narrow: passing the whole record would hand the model fields it
    has no business repeating, and every extra key is a chance for it to state
    something the pipeline never computed. No suggested action is included, so
    the recommendation the model returns is its own reading of these facts rather
    than a rewrite of a keyword rule.
    """
    return {
        "index": index,
        "service": record["service_name"],
        "date": record["display_date"],
        "observed_cost_usd": record["anomaly_value"],
        "baseline_cost_usd": record["normal_value"],
        "increase_pct": record["percentage"],
```

```
        "impact_usd": record["impact_value"],
        "driver_line_item_cost_usd": record["driver_row_cost"],
        "region": record["region"],
        "region_share_pct": record["region_share_pct"],
        "resource": record["resource"],
        "usage_type": record["usage_type"],
        "tag": record["tag"],
        "top_features": record["top_ml_features"],
        "feature_attribution": [
            {
                "feature": item["feature"],
                "value": item["feature_value"],
                "shap_value": item["shap_value"],
            }
            for item in record.get("feature_contributions", [])[:3]
        ],
        "attribution_method": record["attribution_method"],
        "model_flagged": bool(record["model_flag"]),
        "reference_only": bool(record["is_fallback"]),
        "no_prior_baseline": bool(record["is_new_spend"]),
    }


def _post(url: str, body: Dict) -> Dict:
    request = urllib.request.Request(
        url,
        data=json.dumps(body).encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(request, timeout=TIMEOUT_SECONDS) as response:
        return json.loads(response.read().decode("utf-8"))


def _call_gemini(payload: List[Dict], key: str) -> Optional[Dict]:
    """One request, with a single retry for transient 429/500/503 responses."""
    global _LAST_ERROR

    # Quoted rather than interpolated raw: the key goes into a query string, and a
    # stray character in it should produce an API error we can report, not an
    # unreadable InvalidURL from urllib before the request is even sent.
    url = f"{API_ROOT}/{MODEL}:generateContent?key={urllib.parse.quote(key, safe='')}"
    body = {
        "systemInstruction": {"parts": [{"text": SYSTEM_PROMPT}]},
        "contents": [
            {
                "role": "user",
                "parts": [
                    {
                        "text": (
                            "Write an explanation and a recommended action for each of "
                            "these detected cost anomalies:\n\n"
                            + json.dumps(payload, indent=2)
                        )
                    }
                ],
            }
        ],
        "generationConfig": {
            "responseMimeType": "application/json",
            "responseSchema": RESPONSE_SCHEMA,
            # Low temperature: this is a writing task over fixed facts, not a
            # creative one, and drift here reads as invented numbers.
            "temperature": 0.2,
            "maxOutputTokens": MAX_OUTPUT_TOKENS,
```

```
        },
    }

    for attempt in range(2):
        try:
            return _post(url, body)
        except urllib.error.HTTPError as exc:
            detail = exc.read().decode("utf-8", "replace")[:300]
            _LAST_ERROR = f"HTTP {exc.code}: {detail}"
            if exc.code in {429, 500, 503} and attempt == 0:
                time.sleep(2)
                continue
            return None
        except Exception as exc:
            # Network failure, DNS, timeout, TLS: all mean "no LLM text this run",
            # never "break the analysis".
            _LAST_ERROR = f"{type(exc).__name__}: {exc}"
            if attempt == 0:
                time.sleep(2)
                continue
            return None
    return None


def _clean_choice(value: object, allowed: tuple) -> str:
    """Return `value` if it is one of `allowed`, else "".

    The schema constrains these to an enum, but the enum is enforced by the API
    rather than by us, and the UI colours a badge from the result. An unexpected
    string is dropped instead of being rendered.
    """
    text = str(value or "").strip().lower()
    return text if text in allowed else ""


def _clean_causes(value: object) -> List[Dict[str, str]]:
    """Keep only hypotheses that carry both a mechanism and its evidence.

    A cause with no evidence is exactly the unsupported speculation the prompt
    forbids, so it is discarded here rather than shown to the user.
    """
    if not isinstance(value, list):
        return []
    causes = []
    for item in value[:4]:
        if not isinstance(item, dict):
            continue
        cause = str(item.get("cause", "")).strip()
        evidence = str(item.get("evidence", "")).strip()
        if not cause or not evidence:
            continue
        causes.append(
            {
                "cause": cause,
                "confidence": _clean_choice(item.get("confidence"), CONFIDENCE_LEVELS) or "low",
                "evidence": evidence,
            }
        )
    return causes


def _clean_steps(value: object) -> List[str]:
    if not isinstance(value, list):
        return []
    return [str(step).strip() for step in value[:4] if str(step).strip()]
```

```
365
366
367 def generate_explanations(records: List[Dict]) -> Optional[Dict[int, Dict]]:
368     """Return {record index: investigation}, or None.
369
370     Each investigation carries `explanation`, `headline`, `severity`,
371     `likely_causes`, `investigation_steps` and `recommended_action`. Only
372     `explanation` is required; the rest are cleaned to safe empty values so a
373     partial reply degrades the card rather than breaking it.
374
375     None is not an error condition. It is the normal signal that the caller should
376     keep the deterministic template text, and it is what happens whenever the
377     feature is switched off or the API is unreachable.
378     """
379     global _LAST_ERROR
380     _LAST_ERROR = None
381
382     if not records:
383         return None
384     if not is_enabled():
385         _LAST_ERROR = "No API key found (set GEMINI_API_KEY in .env)." if not api_key() else None
386         return None
387
388     key = api_key()
389     # A key pasted across a line break or with a stray space is a common setup
390     # error, and it otherwise surfaces as an opaque URL or transport failure.
391     if any(character.isspace() for character in key):
392         _LAST_ERROR = (
393             "GEMINI_API_KEY contains a space or line break. Check the .env line is "
394             "a single unbroken value."
395         )
396         return None
397
398     payload = [_to_payload(record, i) for i, record in enumerate(records[:MAX_RECORDS_PER_CALL])]
399     response = _call_gemini(payload, key)
400     if response is None:
401         return None
402
403     try:
404         candidate = response["candidates"][0]
405         # A safety block returns 200 with no parts, so the reason is checked first.
406         if candidate.get("finishReason") not in {"STOP", None}:
407             _LAST_ERROR = f"finishReason={candidate.get('finishReason')}"
408             if candidate.get("finishReason") != "MAX_TOKENS":
409                 return None
410         text = candidate["content"]["parts"][0]["text"]
411         parsed = json.loads(text)
412     except Exception as exc:
413         _LAST_ERROR = f"Unparseable reply: {type(exc).__name__}: {exc}"
414         return None
415
416     explanations: Dict[int, Dict] = {}
417     for item in parsed.get("explanations", []):
418         try:
419             index = int(item["index"])
420         except (KeyError, TypeError, ValueError):
421             continue
422         explanation = str(item.get("explanation", "")).strip()
423         if not explanation:
424             # Without the narrative there is nothing to show, so this record keeps
425             # its template text rather than appearing half-filled.
426             continue
427         explanations[index] = {
428             "explanation": explanation,
429             "headline": str(item.get("headline", "")).strip(),
```

```
            "severity": _clean_choice(item.get("severity"), SEVERITY_LEVELS),
            "likely_causes": _clean_causes(item.get("likely_causes")),
            "investigation_steps": _clean_steps(item.get("investigation_steps")),
            "recommended_action": str(item.get("recommended_action", "")).strip(),
        }

    if not explanations:
        _LAST_ERROR = "Reply contained no usable explanations."
        return None
    return explanations
```

10 `carci/reporting.py` — Building the exports

carci/reporting.py

98 lines · Text and CSV exports

10.1 Purpose and concepts

The smallest and simplest file. It converts the analysis result into the two downloadable formats.

`build_report_text()` — note line 7. If the analysis already produced a `report_text` (which `analysis.py` does), it returns that directly. The loop below is only a fallback for results built some other way. This is defensive programming: the function works correctly regardless of which path produced its input.

`build_report_dataframe()` — produces the "Detailed CSV" in long format: one fact per row, with section, metric and value columns, including per-feature SHAP contributions. Long format is harder to read directly but far easier to filter and pivot in Excel, which is what a finance analyst will actually do with it.

Both exports carry whichever explanation the analysis produced. A report downloaded with the LLM enabled contains the model's wording; the same file analysed offline contains the template wording. The numbers are identical either way.

10.2 Complete source code

```
1  from __future__ import annotations
2
3  import pandas as pd
4
5
6  def build_report_text(result: dict) -> str:
7      if "report_text" in result:
8          return result["report_text"]
9
10     lines = ["CARCI Cost Anomaly Root Cause Report", ""]
11     for record in result.get("service_comparison", []):
12         lines.extend(
13             [
14                 f"{record['service_name']} - {record['date']}",
15                 f"Cost: {record['anomaly_value']}",
16                 f"Normal baseline: {record['normal_value']}",
17                 f"Impact: {record['impact_value']}",
18                 f"Region: {record['region']}",
19                 f"Resource: {record['resource']}",
20                 f"Tag: {record.get('tag', 'Untagged')}",
21                 f"Usage type: {record['usage_type']}",
22                 f"Recommendation: {record.get('recommendation', 'Review the contributing factors.')}",
23                 "",
24             ]
25         )
26     return "\n".join(lines)
27
28
29 def build_report_dataframe(result: dict) -> pd.DataFrame:
30     rows = []
31     metadata = result.get("metadata", {})
```

```
    for key, value in metadata.items():
        rows.append({"section": "metadata", "metric": key, "value": value})

    for record in result.get("service_comparison", []):
        rows.extend(
            [
                {"section": record["service_name"], "metric": "date", "value": record["date"]},
                {"section": record["service_name"], "metric": "anomaly_cost", "value":
record["anomaly_value"]},
                {"section": record["service_name"], "metric": "normal_cost", "value":
record["normal_value"]},
                {"section": record["service_name"], "metric": "impact", "value": record["impact_value"]},
                {"section": record["service_name"], "metric": "increase_pct", "value":
record["percentage"]},
                {"section": record["service_name"], "metric": "region", "value": record["region"]},
                {"section": record["service_name"], "metric": "resource", "value": record["resource"]},
                {"section": record["service_name"], "metric": "tag", "value": record.get("tag",
"Untagged")},
                {"section": record["service_name"], "metric": "usage_type", "value":
record["usage_type"]},
                {
                    "section": record["service_name"],
                    "metric": "top_ml_features",
                    "value": ", ".join(record.get("top_ml_features", [])),
                },
            ]
        )

        rows.extend(
            [
                {"section": record["service_name"], "metric": "severity",
                 "value": record.get("severity", "")},
                {"section": record["service_name"], "metric": "headline",
                 "value": record.get("headline", "")},
            ]
        )

        # Ranked, so the row name carries the rank and the export stays sortable.
        for rank, cause in enumerate(record.get("likely_causes", []), start=1):
            rows.append(
                {
                    "section": record["service_name"],
                    "metric": f"likely_cause_{rank}",
                    "value": (
                        f"[{cause.get('confidence', '')}] {cause.get('cause', '')} "
                        f"| evidence: {cause.get('evidence', '')}"
                    ),
                }
            )

        for step_number, step in enumerate(record.get("investigation_steps", []), start=1):
            rows.append(
                {
                    "section": record["service_name"],
                    "metric": f"investigation_step_{step_number}",
                    "value": step,
                }
            )

        for contribution in record.get("feature_contributions", [])[:5]:
            rows.append(
                {
                    "section": record["service_name"],
                    "metric": f"feature_contribution_{contribution['feature']}",
                    "value": (
```

```
                        f"value={contribution['feature_value']}, "
                        f"shap={contribution['shap_value']}"
                    ),
                }
            )

    return pd.DataFrame(rows)
```

11 How the evaluation works

Run it with:

```
python -m carci.isolation_forest_model dataset.csv
```

This calls `run_model()` in `isolation_forest_model.py`, which performs seven steps:

```
dataset.csv  (10,000 rows: 9,000 normal + 1,000 anomaly)
      |
 1.   add_engineered_features()   ->  build the 7 features
      |
 2.   SPLIT   train = 7,200   80% of the NORMAL rows only, no anomalies
              test  = 2,800   1,800 held-out normals + all 1,000 anomalies
      |
 3.   StandardScaler   fit on train, then transform test
      |
 4.   IsolationForest(1000 trees).fit(x_train)
      |
 5.   threshold = 5% quantile of the TRAINING scores      -> 0.000000
      |
 6.   y_pred = decision_function(x_test) < threshold
      |
 7.   compare y_pred against the true labels
      -> Accuracy, Precision, Recall, F1
```

> **Why step 2 is the one that matters**
>
> The anomalies are removed from the training set entirely. The model learns only what normal looks like, and meets all 1,000 anomalies for the first time during testing. If the anomalies were in the training data, the model could memorise them and the scores would be worthless. When you review any ML project, this is the first thing to check.

11.1 The results

Model	Accuracy	Precision	Recall	F1
Isolation Forest	0.845	0.877	0.657	0.751
Cost percentile baseline	0.708	0.665	0.367	0.473
Gradient Boosting (5-fold CV)	1.000	1.000	1.000	1.000

Averaged over 10 different random splits the Isolation Forest scores F1 = 0.748 ± 0.004. That small standard deviation is important: it proves the result is stable and not a lucky split. Always report a spread, never a single number.

11.2 The perfect score is a bug, not an achievement

Label leakage — the most valuable lesson in this project

The Gradient Boosting row scores a flawless 1.000 on every metric. That is not a good model. It is a broken experiment.

The `usage_type` column in this dataset maps **perfectly** onto the label: `NormalUsage`, `BurstUsage`, `BackupWindow` and `BatchJob` are 100% normal, while `SuspiciousSpike`, `CryptoMiningTraffic`, `RunawayJob`, `MassiveDataTransfer`, `UnauthorizedScaling`, `SubtleCostDrift` and `MisconfiguredTagging` are 100% anomaly, with no overlap at all. Gradient Boosting uses that column, so it simply reads the answer instead of detecting anything.

The Isolation Forest is **unaffected**, because `usage_type` is not in `FEATURE_COLUMNS` — it only ever sees the seven numeric features.

> **The lesson: treat a perfect score as a bug report**
>
> Real problems are never solved perfectly. When you see 1.000, go looking for the leak.

11.3 Testing your own design decisions

`daily_z` was added to give the model day-level context. Does it actually help? Testing all four combinations over the same 10 splits:

Features used	Fixed threshold	Calibrated threshold
6 features (no `daily_z`)	0.759	0.760
7 features (with `daily_z`)	0.748	0.748

Two honest conclusions. The threshold change is score-neutral — its value is portability between files, not accuracy. And `daily_z` actually costs about 0.012 F1 on this dataset. It helps when anomalies cluster onto a few days, but this file spreads 1,000 anomalies across 206 days, so daily totals barely move and the feature mostly dilutes the other six.

> **Teaching point**
>
> This result is unflattering, and it is reported anyway. Testing your own ideas and publishing the answer even when it contradicts you is what separates engineering from advocacy. An examiner will trust every other number in a report that contains a result like this one.

12 Known limitations

1. **Recall is 0.657** — roughly a third of anomalies are missed. Be explicit about this rather than letting the F1 score imply more.
2. **SHAP is slow.** About 170 ms per row: 28 minutes for a 10,000-row file, some 99% of total runtime. The app appears frozen for the duration. Explaining only the rows actually displayed would fix it.
3. `contamination = 0.05` **guarantees false positives.** A perfectly clean file still gets 5% of its rows flagged. On `dataset.csv` the app flags 1,107 of 10,000 rows: 450 false positives (5% of the 9,000 normal training rows) plus the 657 anomalies genuinely caught.
4. **The model is fitted on the file it scores** in app mode. Standard for unsupervised detection with no training corpus, but it means the forest sees the anomalies it is hunting.
5. **The test set is 36% anomalies** against the file's true 10%, which inflates precision for every model in the comparison table.
6. **No model persistence.** Nothing is saved to disk; the forest is rebuilt on every single run.
7. **The explanation depends on a third-party API.** The wording the user reads comes from Gemini. The fallback keeps the app working offline, but the two versions do not read alike, so output is not byte-reproducible between runs or between machines.
8. **The fallback is silent.** An expired key or a blocked network produces template text and no error dialog. The caption on page 3 is the only thing that distinguishes them, and a reader who does not know to look will not notice.
9. **Prompt adherence is not enforced.** The system prompt asks for three or four short paragraphs; the model frequently returns one. Nothing validates the returned prose against the rules — only its JSON shape is guaranteed by the schema.
10. **Only the first 20 anomalies are explained.** `MAX_RECORDS_PER_CALL` caps the batch, which is invisible when `MAX_ANOMALIES` is also 20, but would silently leave records unexplained if that cap were raised.

13 Glossary

Term	Meaning
Anomaly	A data point that does not fit the pattern of the rest
Baseline	What is normal. Here: the mean of a service's other days
Contamination	The assumed proportion of anomalies; sets the cut-off
CUR	Cost and Usage Report — AWS's detailed billing export
`decision_function`	The raw score. Low means anomalous
Feature	One number describing a row that the model can use
Feature engineering	Creating new features by combining existing ones
F1 score	The harmonic mean of precision and recall
Graceful degradation	Losing a feature without losing the system, as when the LLM is unreachable and the template is used instead
Label leakage	When a feature secretly contains the answer
LLM	Large language model. Here Gemini, used only to word figures the pipeline computed
Overfitting	Memorising the training data instead of learning the pattern
Precision	Of everything flagged, what fraction was truly anomalous
Prompt	The instructions sent to a language model. Here split into a system prompt of rules and a user message of anomaly facts
`random_state`	A fixed seed so random results reproduce exactly
Recall	Of all true anomalies, what fraction was found
Response schema	A structure the API is required to return, making the reply parseable instead of scraped
SHAP	A method assigning each feature a contribution to one prediction
`StandardScaler`	Rescales features to mean 0, standard deviation 1
Temperature	How much randomness a language model uses when choosing words. 0.2 here
Unsupervised	Learning without labelled answers
z-score	How many standard deviations a value sits from the mean

14 Questions to test understanding

1. Why can the web app not report an accuracy score?
2. Why are anomalies removed from the training set?
3. A clean file with no anomalies still produces flags. Why is that expected?
4. Why is `cost / (usage_quantity + 1)` written with the `+ 1`?
5. Gradient Boosting scored 1.000. Why is that bad news?
6. Why must `ParserError` be caught before `ValueError`?
7. Why compare against a 90th-percentile cost rule at all?
8. Why is comparing a row against its own service's other days better than comparing it against every other row?
9. Precision is 0.877 and recall is 0.657. For a cost alerting tool, is that the right trade-off? Argue both sides.
10. SHAP takes 28 minutes on 10,000 rows. Propose two different fixes and give the drawback of each.
11. The language model is given the anomaly figures but is forbidden from calculating anything. Why is that restriction the single most important rule in the prompt?
12. `_to_payload()` sends nineteen named fields rather than the whole record. Give two distinct reasons why sending less is safer.
13. `generate_explanations()` returns `None` on failure instead of raising. Defend that choice, then argue against it.
14. A user reads a root cause paragraph and finds it useful. What evidence would you need before concluding that the language model produced it?
15. The reply is constrained by a `responseSchema`. What class of bug does that eliminate, and what class does it leave untouched?

Questions 11 to 15 cover the language model layer described in section 9.